LAPORAN PRAKTIKUM PEKAN 6

PEMOGRAMAN BERORIENTASI OBJEK

Abstraction



Oleh:
Naysha Aprilia Rizki
2511531014
Kelas B

Dosen Pengampu:
Afdhal Dinilhak M.Kom

Asisten Praktikum:
Ezza Addini

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, diperlukan suatu konsep pemrograman yang dapat membantu menyederhanakan struktur program agar lebih mudah dipahami, dikembangkan, dan dikelola. Salah satu konsep dalam Pemrograman Berorientasi Objek (PBO) yang dapat digunakan adalah *Abstraction* atau abstraksi. Konsep ini bertujuan untuk menampilkan fungsi atau hal-hal penting dari suatu objek serta menyembunyikan detail proses yang tidak perlu diketahui oleh pengguna.

Dalam Java, abstraksi dapat diterapkan menggunakan *Abstract Class* dan *Interface*. Abstract class digunakan sebagai kerangka dasar bagi class turunannya, sedangkan interface digunakan untuk menentukan kemampuan atau aturan yang harus dimiliki oleh suatu class. Selain itu, abstraksi juga dapat diterapkan melalui penggunaan *default method*, *static method*, *private method*, *interface inheritance*, serta kata kunci final.

Pada praktikum ini, konsep abstraksi diterapkan dalam pengembangan sistem toko online UMKM. Program dirancang dengan beberapa fitur, seperti pengelolaan produk, pengiriman, pembayaran, diskon, dan pesanan. Melalui praktikum ini, mahasiswa dapat memahami bagaimana abstraksi membantu menghasilkan program yang lebih terstruktur, fleksibel, dan mudah dikembangkan.

1.2 Tujuan

1. Memahami konsep *Abstraction* dalam Pemrograman Berorientasi Objek menggunakan bahasa Java.
2. Mengetahui perbedaan serta fungsi *Abstract Class* dan *Interface*.

3. Mampu menerapkan abstract class dan abstract method dalam pembuatan program.
4. Memahami penggunaan method pada interface, seperti *default method*, *static method*, dan *private method*.
5. Mampu menerapkan konsep abstraksi dalam merancang program yang terstruktur, fleksibel, dan mudah dikembangkan.

1.3 Manfaat

1. Menambah pemahaman mengenai konsep abstraksi dalam pemrograman berorientasi objek.
2. Mempermudah dalam menyusun kode program agar lebih terstruktur dan mudah dipahami.
3. Meningkatkan kemampuan dalam menggunakan abstract class dan interface pada Java.
4. Membantu mengurangi ketergantungan antarclass sehingga program lebih fleksibel untuk dikembangkan.
5. Melatih kemampuan merancang dan membangun aplikasi Java melalui penerapan konsep abstraksi.

BAB II

PEMBAHASAN

1. Teori

Abstraksi (*Abstraction*) merupakan salah satu konsep dalam Pemrograman Berorientasi Objek yang berfungsi menyederhanakan program dengan menampilkan bagian penting dari suatu objek dan menyembunyikan detail implementasi yang tidak perlu diketahui. Dalam Java, abstraksi dapat diterapkan menggunakan *Abstract Class* dan *Interface*. Pada praktikum ini, konsep abstraksi diterapkan dalam pembuatan sistem toko online UMKM.

1) Abstraksi (*Abstraction*)

Abstraksi merupakan konsep yang berfokus pada fungsi atau karakteristik utama suatu objek tanpa menampilkan seluruh proses yang terjadi di dalamnya. Dalam Java, abstraksi membantu membuat program lebih sederhana, terstruktur, dan mudah dikembangkan.

2) Abstract Class

Abstract Class merupakan kelas yang tidak dapat dibuat objeknya secara langsung dan digunakan sebagai kerangka dasar bagi kelas turunannya. Abstract class dapat memiliki atribut, constructor, method biasa, maupun *abstract method*. Dalam praktikum ini, konsep tersebut digunakan untuk membentuk struktur dasar objek yang memiliki karakteristik atau perilaku serupa.

3) Interface

Interface merupakan kumpulan aturan atau kemampuan yang harus diterapkan oleh kelas yang mengimplementasikannya. Interface membantu memisahkan definisi kemampuan dengan proses pelaksanaannya. Dalam sistem toko online, interface dapat digunakan untuk menentukan kemampuan tertentu, seperti proses pembayaran, pengiriman, dan pemberian diskon.

2. Langkah Kerja.

1) Pembuatan Folder untuk menyimpan folder dan class yang berisikan kode program java di VS Code.
2) Buat package yang Bernama “Shop” untuk menyiapkan beberapa package yaitu app, discount, model, payment, service, dan shipping.
3) Masukan beberapa package yang berguna untuk mendukung praktikum PBO pekan 6 ini, dan package yang dimasukan di dapat dari asisten praktikum. Package yang ditambah yaitu review, selfcheck, tools dan clas studentinfo.

4) Membuat kode program yang terdapat penambahan atau perubahan di dalam modul praktikum:

1. Latihan

Pada bagian src itu terdapat 4 folder yaitu review, selfcheck, shop, tools.

a. review : berisi BadShop.java (contoh program toko dengan desain yang kurang baik. Digunakan sebagai bahan perbandingan atau evaluasi desain.)
b. selfcheck : berisi SelfCheck.java (memeriksa apakah kode latihan sudah sesuai dengan ketentuan modul.) dan TugasCheck.java (memeriksa apakah pengerjaan tugas individu sudah sesuai spesifikasi.)
c. shop : berisi app, discount, model, payment, service, shipping (akan dijelaskan lebih lanjut di point selanjutnya)
d. tools : berisi main.java (program utama untuk menjalankan atau membantu proses pengujian sesuai isi kodenya.) dan NimParams.java (menghitung parameter khusus tugas berdasarkan NIM mahasiswa.)

1) Latihan 1 Abstract class dan abstract method

a. Class Product

Latihan > abstraction-2511531014 > src > shop > model > J Product.java > {} shop.model

```java
package shop.model;

public abstract class Product {
    private final String sku;
    private final String name;
    private final long price; // in rupiah

    protected Product(String sku, String name, long price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU must not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price must not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() { return sku; }
    public String getName() { return name; }
    public long getPrice() { return price; }

    /** Price the customer actually pays. Subclasses may override it. */
    public long getFinalPrice() { return price; }

    /** Every concrete product must state its category. */
    public abstract String category();

    /** A concrete method that relies on the abstract method category(). */
    public String getInfo() {
        return "[" + category() + "] " + name + " - Rp" + getFinalPrice();
    }
}
```

Kode ini mendefinisikan class Product di package shop.model sebagai class abstract, sehingga tidak bisa langsung dibuat objeknya dan hanya

menjadi dasar bagi class turunan seperti produk makanan atau elektronik. Atributnya, yaitu sku, name, dan price (dalam rupiah), bersifat private final, jadi tidak bisa diakses langsung dari luar class dan nilainya tidak bisa diubah setelah objek dibuat. Constructor-nya protected sehingga hanya bisa dipanggil oleh subclass, dan di dalamnya ada validasi: SKU dan nama tidak boleh null atau kosong, serta harga tidak boleh negatif, kalau dilanggar akan melempar IllegalArgumentException. Setelah lolos validasi, nilai parameter disimpan ke atribut menggunakan this. Getter getSku(), getName(), dan getPrice() dipakai untuk membaca data, sedangkan getFinalPrice() mengembalikan harga yang benar-benar dibayar pelanggan dan bisa di-override subclass, misalnya untuk menerapkan diskon. Method category() dideklarasikan abstract tanpa isi, sehingga setiap subclass konkret wajib menentukan kategorinya sendiri. Terakhir, getInfo() adalah method konkret yang memanggil category() dan getFinalPrice() untuk menghasilkan teks informasi produk berformat [kategori] nama - Rp harga, dan ini merupakan penerapan polimorfisme karena hasilnya menyesuaikan implementasi subclass yang digunakan.

b. Class PhysicalProduct

```java
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {

    private final int weightGrams;

    public PhysicalProduct(String sku, String name, long price, int weightGrams) {
        super(sku, name, price);

        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight must be positive");
        }

        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() {
        return weightGrams;
    }

    @Override
    public String category() {
        return "Fisik";
    }
}
```

Kode ini mendefinisikan class PhysicalProduct di package shop.model yang merupakan subclass dari Product (lewat extends) sekaligus

mengimplementasikan interface Shippable yang diimpor dari package shop.shipping, sehingga produk fisik ini dapat dikirim. Class ini menambahkan satu atribut baru, yaitu weightGrams (berat dalam gram) yang bersifat private final, jadi tidak bisa diakses langsung dari luar dan nilainya tidak bisa diubah setelah objek dibuat. Constructor-nya bersifat public dan menerima sku, name, price, serta weightGrams. Tiga parameter pertama diteruskan ke constructor class induk melalui super(sku, name, price), sehingga validasi SKU, nama, dan harga tetap dilakukan oleh Product. Setelah itu, constructor memvalidasi berat: jika weightGrams kurang dari atau sama dengan 0, akan dilempar IllegalArgumentException dengan pesan "Weight must be positive", dan jika lolos nilainya disimpan ke atribut melalui this.weightGrams. Method getWeightGrams() yang diberi anotasi @Override merupakan implementasi dari method di interface Shippable dan berfungsi mengembalikan berat produk. Terakhir, method category() juga di-@Override untuk mengimplementasikan method abstract dari Product, dengan mengembalikan teks "Fisik" sebagai kategori produk, sehingga class ini sudah menjadi class konkret yang bisa diinstansiasi dan hasil getInfo()-nya akan menampilkan kategori [Fisik].

c. Class DigitalProduct

```java
package shop.model;

public class DigitalProduct extends Product {
    public String downloadUrl;

    public DigitalProduct(String sku, String name, long price, String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank()) {
            throw new IllegalArgumentException("Download URL must not be blank");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
    public String category() {
        return "Digital";
    }
}
```

Kode ini mendefinisikan class DigitalProduct di package shop.model yang merupakan subclass dari Product (lewat extends), sehingga mewarisi atribut dan method milik Product seperti getSku(), getName(),

getPrice(), getFinalPrice(), dan getInfo(). Class ini menambahkan satu atribut baru, yaitu downloadUrl bertipe String yang bersifat public, artinya bisa diakses dan diubah langsung dari luar class. Constructor-nya bersifat public dan menerima sku, name, price, serta downloadUrl. Tiga parameter pertama diteruskan ke constructor class induk melalui super(sku, name, price), sehingga validasi SKU, nama, dan harga tetap dilakukan oleh Product. Setelah itu, constructor memvalidasi downloadUrl: jika bernilai null atau kosong, akan dilempar IllegalArgumentException dengan pesan "Download URL must not be blank", dan jika lolos nilainya disimpan ke atribut melalui this.downloadUrl. Method getDownloadUrl() berfungsi sebagai getter untuk membaca alamat unduhan produk. Terakhir, method category() diberi anotasi @Override untuk mengimplementasikan method abstract dari Product dengan mengembalikan teks "Digital", sehingga class ini menjadi class konkret yang bisa diinstansiasi dan hasil getInfo()-nya menampilkan kategori [Digital].

Kode ini mendefinisikan class DigitalProduct di package shop.model yang merupakan subclass dari Product (lewat extends) sekaligus mengimplementasikan interface Discountable yang diimpor dari package shop.discount, sehingga produk digital ini dapat diberi diskon. Class ini memiliki konstanta DIGITAL_DISCOUNT_PERCENT bernilai 10 yang bersifat private static final, artinya nilainya tetap, dimiliki oleh class (bukan per objek), dan hanya bisa diakses dari dalam class, serta satu atribut downloadUrl bertipe String yang bersifat private final sehingga tidak bisa diubah setelah objek dibuat. Constructor-nya bersifat public dan menerima sku, name, price, serta downloadUrl. Tiga parameter pertama diteruskan ke constructor class induk melalui super(sku, name, price), sehingga validasi SKU, nama, dan harga tetap dilakukan oleh Product. Setelah itu, constructor memvalidasi downloadUrl: jika bernilai null atau kosong, akan dilempar

IllegalArgumentException dengan pesan “Download URL must not be blank”, dan jika lolos nilainya disimpan ke atribut melalui this.downloadUrl. Method getDownloadUrl() berfungsi sebagai getter untuk membaca alamat unduhan produk. Method discountPercent() yang diberi anotasi @Override merupakan implementasi dari interface Discountable dan mengembalikan persentase diskon sebesar 10. Method getFinalPrice() juga di-@Override untuk menggantikan versi milik Product, dengan memanggil applyDiscount(getPrice()) sehingga harga yang dibayar pelanggan sudah dipotong diskon. Terakhir, method category() di-@Override untuk mengimplementasikan method abstract dari Product dengan mengembalikan teks “Digital”, sehingga class ini menjadi class konkret yang bisa diinstansiasi dan hasil getInfo()-nya akan menampilkan kategori [Digital] beserta harga setelah diskon.

d. Class ShippingService

```java
package shop.shipping;

public abstract class ShippingService {

    private final String name;

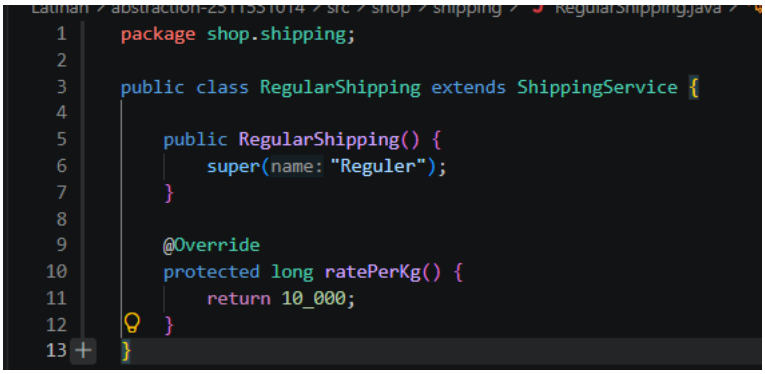
    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
    }

    public String getName() {
        return name;
    }
```

Kode ini mendefinisikan class ShippingService di package shop.shipping sebagai class abstract, sehingga tidak bisa langsung dibuat objeknya dan hanya menjadi kerangka dasar bagi layanan pengiriman tertentu (misalnya reguler atau kilat). Class ini memiliki atribut name (nama layanan pengiriman) yang bersifat private final, jadi tidak bisa diakses langsung dari luar dan nilainya tidak bisa diubah setelah objek dibuat. Constructor-nya bersifat protected sehingga hanya bisa dipanggil oleh subclass, dan di dalamnya dilakukan validasi: jika name bernilai null atau kosong, akan dilempar IllegalArgumentException dengan pesan "Name must not be blank", dan jika lolos nilainya disimpan ke atribut melalui this.name. Method

getName() berfungsi sebagai getter untuk membaca nama layanan. Method ratePerKg() dideklarasikan protected abstract tanpa isi, sehingga setiap subclass wajib menentukan tarif per kilogramnya sendiri (dalam rupiah) dan method ini hanya bisa diakses oleh class turunan.

e. Class RegularShipping dan ExpressShipping



```java
package shop.shipping;

public class RegularShipping extends ShippingService {

    public RegularShipping() {
        super(name: "Reguler");
    }

    @Override
    protected long ratePerKg() {
        return 10_000;
    }
}
```

Kode ini mendefinisikan class RegularShipping di package shop.shipping yang merupakan subclass dari ShippingService (lewat extends), sehingga class ini mewarisi atribut name, method getName(), dan method calculateCost() dari class induknya. Constructor-nya bersifat public dan tidak menerima parameter, karena nama layanan sudah ditetapkan langsung lewat super("Reguler"), sehingga setiap objek RegularShipping otomatis bernama "Reguler" dan tetap melewati validasi nama yang ada di ShippingService. Method ratePerKg() diberi anotasi @Override untuk mengimplementasikan method abstract dari class induk, dengan mengembalikan tarif 10_000 (Rp10.000 per kilogram, tanda underscore hanya pemisah agar angka lebih mudah dibaca). Method ini bersifat protected sesuai deklarasi di induknya, sehingga hanya dipakai secara internal oleh calculateCost(). Dengan begitu, class ini menjadi class konkret yang bisa diinstansiasi, dan biaya pengiriman reguler dihitung dari berat yang dibulatkan ke atas dalam kilogram lalu dikalikan Rp10.000, misalnya 700 gram dikenai Rp10.000 dan 1.500 gram dikenai Rp20.000. Berbeda dengan ExpressShipping, class ini memakai tarif yang lebih murah, sehingga

keduanya menunjukkan polimorfisme: cara hitung di calculateCost() sama, tetapi hasilnya berbeda sesuai tarif masing-masing subclass.



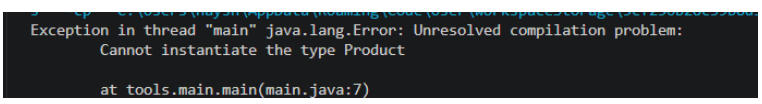
```java
package shop.shipping;

public class ExpressShipping extends ShippingService {
    public ExpressShipping() {
        super(name: "Ekspres");
    }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }
}
```

Kode ini mendefinisikan class ExpressShipping di package shop.shipping yang merupakan subclass dari ShippingService (lewat extends), sehingga class ini mewarisi atribut name, method getName(), dan method calculateCost() dari class induknya. Constructor-nya bersifat public dan tidak menerima parameter, karena nama layanan sudah ditetapkan langsung lewat super("Ekspres"), sehingga setiap objek ExpressShipping otomatis bernama "Ekspres" dan tetap melewati validasi nama yang ada di ShippingService. Method ratePerKg() diberi anotasi @Override untuk mengimplementasikan method abstract dari class induk, dengan mengembalikan tarif 18_000 (Rp18.000 per kilogram, tanda underscore hanya pemisah agar angka lebih mudah dibaca). Method ini bersifat protected sesuai deklarasi di induknya, sehingga hanya dipakai secara internal oleh calculateCost(). Dengan begitu, class ini menjadi class konkret yang bisa diinstansiasi, dan biaya pengiriman ekspres dihitung dari berat yang dibulatkan ke atas dalam kilogram lalu dikalikan Rp18.000, misalnya 700 gram dikenai Rp18.000 dan 1.500 gram dikenai Rp36.000.

f. Uji coba di class main



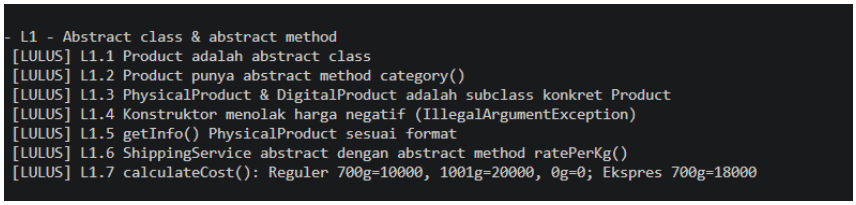
```
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
        Cannot instantiate the type Product

        at tools.main.main(main.java:7)
```

Error ini muncul karena kode mencoba membuat objek langsung dari class Product menggunakan Product p = new Product("X", "x", 10000);. Padahal Product dideklarasikan sebagai class abstract, dan class abstract tidak bisa diinstansiasi langsung.

g. Selfcheck

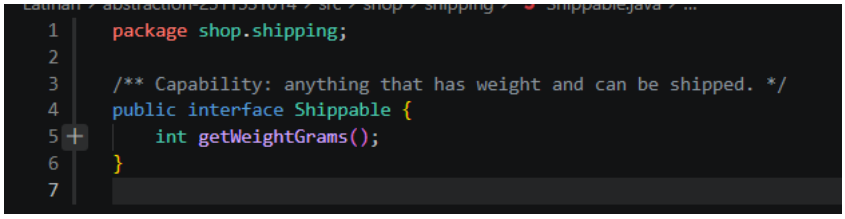
Setelah melakukan semua Langkah diatas lakukan uji coba di bagian selfcheck melihat dibagian mana saja yang gagal atau lulus. Output yangg di hasil kan saat menjalankan selfcheck:



Semua Latihan 1 berhasil di jalankan atau lulus.

2) Latihan 2 (Interface dan program to an interface)

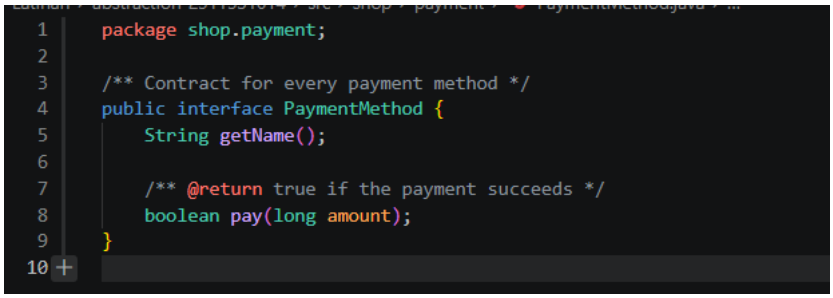
a. Interface Shippable



Kode ini mendefinisikan Shippable di package shop.shipping sebagai interface, yaitu kontrak yang hanya berisi deklarasi method tanpa implementasi, sehingga tidak bisa diinstansiasi langsung dan harus diimplementasikan oleh class lain lewat kata kunci implements. Interface ini merepresentasikan kemampuan (capability) bahwa suatu objek memiliki berat dan dapat dikirim. Di dalamnya hanya ada satu method, yaitu getWeightGrams() yang mengembalikan berat dalam gram bertipe int. Method pada interface otomatis bersifat public dan abstract, sehingga class yang mengimplementasikannya wajib menyediakan isinya, seperti yang dilakukan PhysicalProduct dengan mengembalikan nilai weightGrams. Nilai berat ini kemudian dapat

dipakai oleh ShippingService.calculateCost() untuk menghitung ongkos kirim. Dengan adanya interface ini, kemampuan "dapat dikirim" hanya dimiliki oleh produk yang memang membutuhkannya, sedangkan produk seperti DigitalProduct tidak perlu mengimplementasikannya karena tidak memiliki berat fisik.

b. Interface PaymentMethod



```java
package shop.payment;

/** Contract for every payment method */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);
}
```

Kode ini mendefinisikan PaymentMethod di package shop.payment sebagai interface, yaitu kontrak yang hanya berisi deklarasi method tanpa implementasi, sehingga tidak bisa diinstansiasi langsung dan harus diimplementasikan oleh class lain lewat kata kunci implements. Interface ini menjadi aturan dasar bagi setiap metode pembayaran, misalnya transfer bank, e-wallet, atau kartu kredit. Di dalamnya terdapat dua method. Pertama, getName() yang mengembalikan nama metode pembayaran dalam bentuk String. Kedua, pay(long amount) yang menerima jumlah pembayaran dalam rupiah dan mengembalikan nilai boolean, yaitu true jika pembayaran berhasil dan false jika gagal, sebagaimana dijelaskan pada komentar Javadoc di atasnya. Method pada interface otomatis bersifat public dan abstract, sehingga setiap class yang mengimplementasikannya wajib menyediakan isi kedua method tersebut dengan caranya sendiri. Dengan adanya interface ini, kode lain seperti proses checkout cukup bergantung pada PaymentMethod tanpa perlu tahu jenis pembayaran yang dipakai, sehingga metode pembayaran baru dapat ditambahkan tanpa mengubah kode yang sudah ada, dan ini merupakan penerapan polimorfisme.

c. Class EWalletPayment

```java
package shop.payment;

public class EWalletPayment implements PaymentMethod {

    private String name;
    private long balance;

    public EWalletPayment(String name, long balance) {
        this.name = name;
        this.balance = balance;
    }

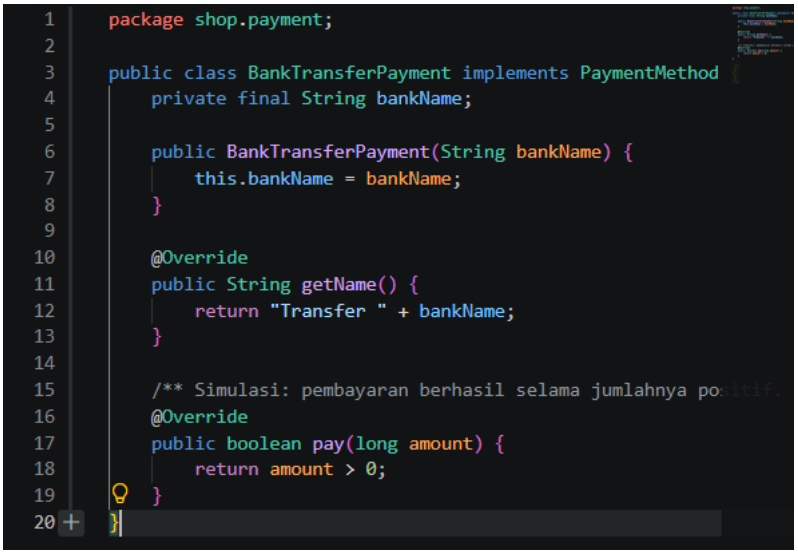
    @Override
    public String getName() {
        return name;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }

        balance -= amount;
        return true;
    }
    public long getBalance() {
        return balance;
    }
}
```

Kode ini mendefinisikan class EWalletPayment di package shop.payment yang mengimplementasikan interface PaymentMethod (lewat implements), yaitu metode pembayaran berupa dompet digital yang saldonya dipakai untuk membayar. Class ini memiliki dua atribut private, yaitu name (nama e-wallet) dan balance (saldo dalam rupiah, bertipe long), sehingga tidak bisa diakses langsung dari luar class. Atribut ini tidak bersifat final karena nilai saldo harus berubah setiap kali terjadi pembayaran. Constructor-nya bersifat public, menerima name dan balance, lalu menyimpannya ke atribut melalui this. Method getName() yang diberi anotasi @Override merupakan implementasi dari interface dan mengembalikan nama e-wallet. Method pay() juga di-@Override dan memproses pembayaran: jika amount kurang dari atau sama dengan 0, atau lebih besar dari saldo, pembayaran ditolak dengan mengembalikan false. Jika valid, saldo dikurangi sebesar amount dan method mengembalikan true. Terakhir, getBalance() adalah getter untuk membaca saldo saat ini. Karena hanya mengimplementasikan PaymentMethod, class ini dapat dipakai lewat tipe PaymentMethod yang sama dengan BankTransferPayment, sehingga ini merupakan penerapan polimorfisme.

d. Class BankTransferPayment



```java
package shop.payment;

public class BankTransferPayment implements PaymentMethod {
    private final String bankName;

    public BankTransferPayment(String bankName) {
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    /** Simulasi: pembayaran berhasil selama jumlahnya positif. */
    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }
}
```

Kode ini mendefinisikan class BankTransferPayment di package shop.payment yang mengimplementasikan interface PaymentMethod (lewat implements), yaitu metode pembayaran berupa transfer bank. Class ini memiliki satu atribut bankName (nama bank) yang bersifat private final, jadi tidak bisa diakses langsung dari luar class dan nilainya tidak bisa diubah setelah objek dibuat. Constructor-nya bersifat public, menerima bankName, lalu menyimpannya ke atribut melalui this. Method getName() yang diberi anotasi @Override merupakan implementasi dari interface dan mengembalikan teks gabungan "Transfer " dengan nama bank, misalnya "Transfer BCA". Method pay() juga di-@Override dan hanya mensimulasikan pembayaran, sebagaimana dijelaskan pada komentar di atasnya: pembayaran dianggap berhasil (true) selama amount lebih besar dari 0, dan gagal (false) jika sebaliknya, tanpa ada pengecekan saldo atau proses transfer yang sebenarnya. Berbeda dengan EWalletPayment, class ini hanya mengimplementasikan PaymentMethod sehingga tidak memiliki method refund(), artinya pembayaran lewat transfer bank dalam program ini tidak mendukung pengembalian dana. Meski begitu, keduanya dapat dipakai lewat tipe PaymentMethod yang sama, sehingga ini merupakan penerapan polimorfisme.

e. Class CheckoutService



```java
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) { // a capability, not a class
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items, ShippingService shipping) {
        return subtotal(items) + shipping.calculateCost(totalWeight(items));
    }

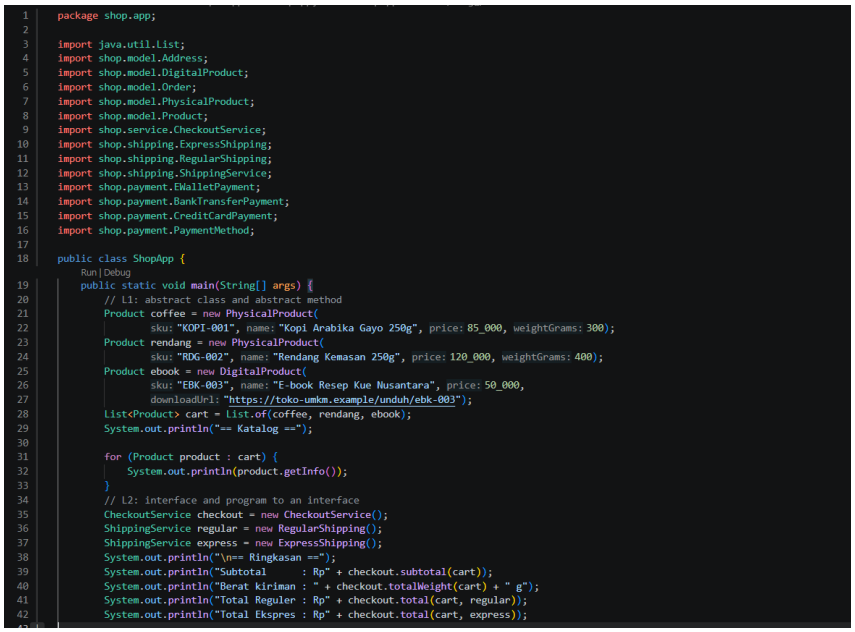
    public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
        long totalAmount = total(items, shipping);
        return payment.pay(totalAmount);
    }

}
```

Kode ini mendefinisikan class CheckoutService di package shop.service yang bertugas mengatur proses checkout: menghitung total belanja, berat barang, biaya kirim, dan pembayaran. Class ini tidak memiliki atribut dan constructor khusus, sehingga hanya berisi method-method layanan yang memanfaatkan class dan interface lain seperti Product, ShippingService, PaymentMethod, RefundablePayment, dan Shippable yang diimpor di bagian atas. Method subtotal() menerima List<Product>, lalu menjumlahkan getFinalPrice() dari setiap produk lewat perulangan for, sehingga harga yang dihitung mengikuti aturan harga akhir masing-masing subclass. Method totalWeight() menjumlahkan berat semua barang yang dapat dikirim. Pengecekan dilakukan dengan instanceof Shippable shippable, yaitu pattern matching yang sekaligus memeriksa tipe dan mengubah variabel menjadi Shippable, sehingga produk digital yang tidak mengimplementasikan Shippable otomatis dilewati. Method total() menjumlahkan subtotal() dengan biaya kirim hasil shipping.calculateCost() berdasarkan total berat, dan karena parameternya bertipe ShippingService, biayanya menyesuaikan jenis pengiriman yang dipilih (reguler atau ekspres). Terakhir, method checkout() menghitung total pembayaran lewat total(), lalu memanggil payment.pay(totalAmount) dan mengembalikan true jika pembayaran berhasil. Karena parameternya bertipe PaymentMethod, class ini bisa

bekerja dengan berbagai metode pembayaran tanpa perlu tahu jenisnya, sehingga ini merupakan penerapan polimorfisme.

f. Class ShopApp



```java
package shop.app;

import java.util.List;
import shop.model.Address;
import shop.model.DigitalProduct;
import shop.model.Order;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;
import shop.payment.EWalletPayment;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.PaymentMethod;

public class ShopApp {
    Run|Debug
    public static void main(String[] args) {
        // L1: abstract class and abstract method
        Product coffee = new PhysicalProduct(
                sku: "KOPI-001", name: "Kopi Arabika Gayo 250g", price: 85_000, weightGrams: 300);
        Product rendang = new PhysicalProduct(
                sku: "RDG-002", name: "Rendang Kemasan 250g", price: 120_000, weightGrams: 400);
        Product ebook = new DigitalProduct(
                sku: "EBK-003", name: "E-book Resep Kue Nusantara", price: 50_000,
                downloadUrl: "https://toko-umkm.example/unduh/ebk-003");
        List<Product> cart = List.of(coffee, rendang, ebook);
        System.out.println("== Katalog ==");

        for (Product product : cart) {
            System.out.println(product.getInfo());
        }
        // L2: interface and program to an interface
        CheckoutService checkout = new CheckoutService();
        ShippingService regular = new RegularShipping();
        ShippingService express = new ExpressShipping();
        System.out.println("\n== Ringkasan ==");
        System.out.println("Subtotal      : Rp" + checkout.subtotal(cart));
        System.out.println("Berat kiriman : " + checkout.totalWeight(cart) + " g");
        System.out.println("Total Reguler : Rp" + checkout.total(cart, regular));
        System.out.println("Total Ekspres : Rp" + checkout.total(cart, express));
```

Kode ini mendefinisikan class ShopApp di package shop.app yang berisi method main sebagai titik awal program. Di bagian atas, import dipakai untuk memanggil class dari package lain, yaitu List dari java.util, class model (Address, DigitalProduct, Order, PhysicalProduct, Product), CheckoutService, class pengiriman (ExpressShipping, RegularShipping, ShippingService), serta class pembayaran (EWalletPayment, BankTransferPayment, CreditCardPayment, PaymentMethod). Di dalam main, dibuat tiga produk: coffee dan rendang sebagai PhysicalProduct (dengan berat 300 g dan 400 g), serta ebook sebagai DigitalProduct yang dilengkapi URL unduhan. Ketiganya disimpan dalam variabel bertipe Product (abstract class), lalu dimasukkan ke List bernama cart. Perulangan for kemudian memanggil getInfo() pada tiap produk untuk menampilkan katalog, dan hasilnya menyesuaikan kategori serta harga masing-masing subclass, sehingga ini penerapan polimorfisme. Selanjutnya dibuat objek CheckoutService, serta dua layanan pengiriman, RegularShipping dan ExpressShipping, yang keduanya disimpan dalam tipe ShippingService

(pemrograman ke tipe induk/interface). Bagian ringkasan lalu mencetak subtotal belanja, total berat kiriman (hanya dari produk fisik), serta total biaya dengan pengiriman reguler dan ekspres, yang berbeda karena tarif per kilogram tiap layanan berbeda.

Output setelah di jalankan:

```
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal      : Rp250000
Berat kiriman : 700 g
Total Reguler : Rp260000
Total Ekspres : Rp268000
```

g. Selfcheck

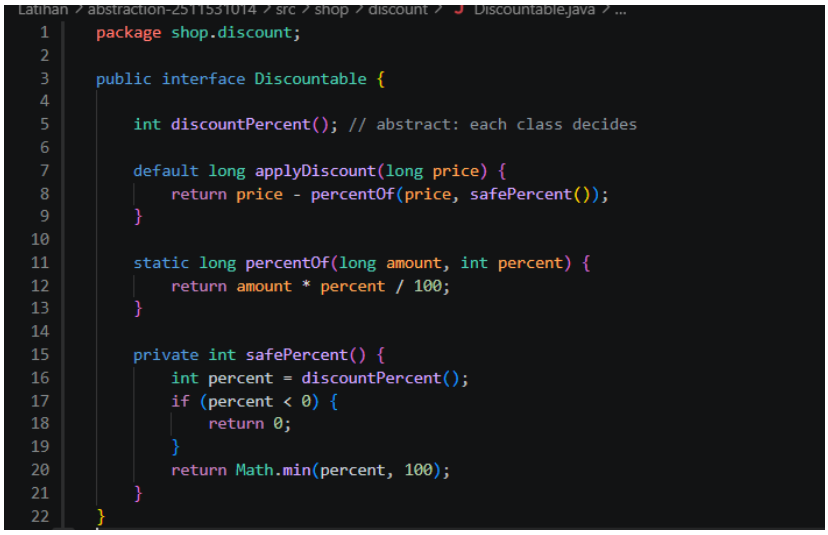
Setelah melakukan semua Langkah diatas lakukan uji coba di bagian selfcheck melihat dibagian mana saja yang gagal atau lulus. Output yangg di hasil kan saat menjalankan selfcheck:

```
-- L2 - Interface & program to an interface
 [LULUS] L2.1 Shippable interface; PhysicalProduct implements, DigitalProduct tidak
 [LULUS] L2.2 PaymentMethod interface dengan getName() dan pay(long)
 [LULUS] L2.3 EWalletPayment: bayar sukses mengurangi saldo, saldo kurang ditolak
 [LULUS] L2.4 CheckoutService.checkout(...) menerima PaymentMethod & ShippingService (bukan class konkret)
 [LULUS] L2.5 Total keranjang contoh: Reguler=260000, Ekspres=268000
```

Semua Latihan 2 berhasil di jalankan atau lulus. (seharusnya L2.5 itu error dan akan jadi lulus saat telah dilaksanakan L3 tapi saat pembuatan laporan ini L3 telah dikerjakan sehingga output menjadi seperti itu)

3) Latihan 3 (Interface lanjutan)

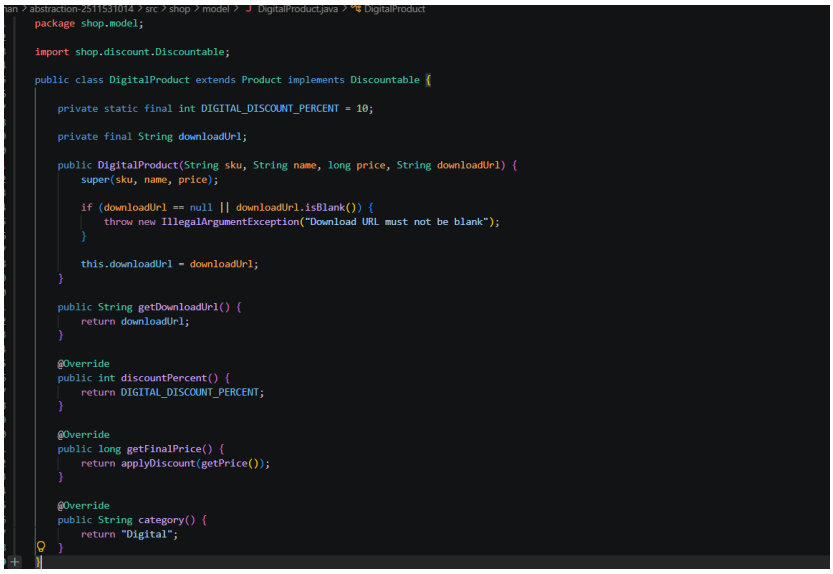
a. Interface Discountable



Kode ini mendefinisikan Discountable di package shop.discount sebagai interface, yaitu kontrak yang merepresentasikan kemampuan (capability) suatu objek untuk diberi diskon, seperti DigitalProduct pada versi yang mengimplementasikannya. Berbeda dengan interface sebelumnya yang hanya berisi deklarasi method, interface ini memuat empat jenis method sekaligus. Pertama, discountPercent() adalah method abstract tanpa isi, sehingga setiap class yang mengimplementasikannya wajib menentukan persentase diskonnya sendiri. Kedua, applyDiscount(long price) adalah default method, yaitu method yang sudah memiliki isi di dalam interface (ditandai kata kunci default) sehingga class pengimplementasi langsung mewarisinya tanpa harus menulis ulang, tetapi tetap boleh di-override. Method ini mengembalikan harga setelah dikurangi diskon, yaitu price dikurangi hasil percentOf(price, safePercent()). Ketiga, percentOf(long amount, int percent) adalah static method, yaitu method milik interface itu sendiri (bukan milik objek) yang dipanggil tanpa membuat objek, dan bertugas menghitung nilai persentase dengan rumus amount * percent / 100. Keempat, safePercent() adalah private method (didukung sejak Java 9) yang hanya bisa dipakai di dalam interface ini sebagai method bantu. Method ini mengambil

persentase dari discountPercent() lalu membatasinya agar aman: jika nilainya negatif dikembalikan 0, dan jika lebih dari 100 dibatasi menjadi 100 lewat Math.min(percent, 100), sehingga harga tidak mungkin naik karena diskon negatif maupun menjadi minus karena diskon lebih dari 100%. Sebagai contoh, harga Rp50.000 dengan diskon 10% menghasilkan percentOf sebesar Rp5.000, sehingga applyDiscount mengembalikan Rp45.000. Dengan adanya interface ini, logika perhitungan diskon cukup ditulis sekali dan dapat dipakai bersama oleh semua class yang mengimplementasikannya, sehingga ini juga menunjukkan penerapan polimorfisme.

b. Class DigitalProduct ikut promo



```java
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements Discountable {

    private static final int DIGITAL_DISCOUNT_PERCENT = 10;

    private final String downloadUrl;

    public DigitalProduct(String sku, String name, long price, String downloadUrl) {
        super(sku, name, price);

        if (downloadUrl == null || downloadUrl.isBlank()) {
            throw new IllegalArgumentException("Download URL must not be blank");
        }

        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }

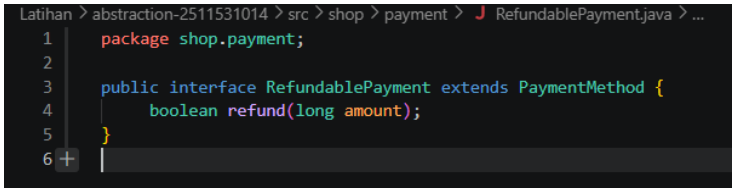
    @Override
    public String category() {
        return "Digital";
    }
}
```

Kode ini mendefinisikan class DigitalProduct di package shop.model yang merupakan subclass dari Product (lewat extends) sekaligus mengimplementasikan interface Discountable yang diimpor dari package shop.discount, sehingga produk digital ini dapat diberi diskon. Class ini memiliki konstanta DIGITAL_DISCOUNT_PERCENT bernilai 10 yang bersifat private static final, artinya nilainya tetap, dimiliki oleh class (bukan per objek), dan hanya bisa diakses dari dalam class, serta satu atribut downloadUrl bertipe String yang bersifat private final sehingga tidak bisa diakses langsung dari luar dan tidak bisa diubah setelah objek

dibuat. Constructor-nya bersifat public dan menerima sku, name, price, serta downloadUrl. Tiga parameter pertama diteruskan ke constructor class induk melalui super(sku, name, price), sehingga validasi SKU, nama, dan harga tetap dilakukan oleh Product. Setelah itu, constructor memvalidasi downloadUrl: jika bernilai null atau kosong, akan dilempar IllegalArgumentException dengan pesan "Download URL must not be blank", dan jika lolos nilainya disimpan ke atribut melalui this.downloadUrl. Method getDownloadUrl() berfungsi sebagai getter untuk membaca alamat unduhan produk. Method discountPercent() yang diberi anotasi @Override merupakan implementasi dari method abstract di interface Discountable dan mengembalikan persentase diskon sebesar 10. Method getFinalPrice() juga di-@Override untuk menggantikan versi milik Product, dengan memanggil applyDiscount(getPrice()) sehingga harga yang dibayar pelanggan sudah dipotong diskon, misalnya Rp50.000 menjadi Rp45.000. Terakhir, method category() di-@Override untuk mengimplementasikan method abstract dari Product dengan mengembalikan teks "Digital", sehingga class ini menjadi class konkret yang bisa diinstansiasi dan hasil getInfo()-nya menampilkan kategori [Digital] beserta harga setelah diskon, yang merupakan penerapan polimorfisme karena hasilnya menyesuaikan implementasi subclass.

c. Interface inheritance

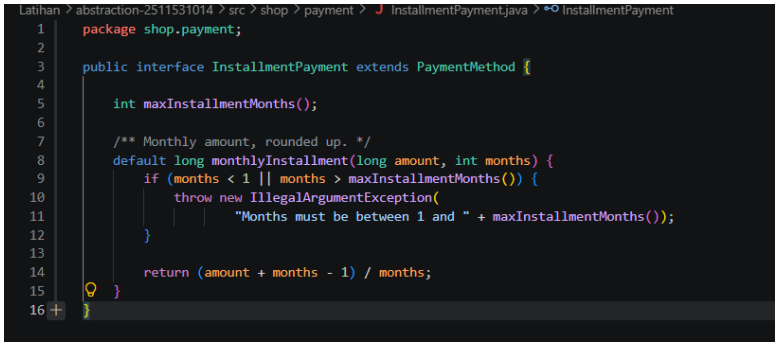
- Interface RefaundablePayment



Kode ini mendefinisikan RefundablePayment di package shop.payment sebagai interface yang memperluas PaymentMethod lewat kata kunci extends, sehingga seluruh aturan milik

PaymentMethod (yaitu getName() dan pay(long amount)) otomatis ikut diwarisi. Ini disebut interface inheritance, yaitu satu interface dibangun di atas interface lain. Interface ini hanya menambahkan satu method abstract, yaitu refund(long amount), yang menerima jumlah dana dalam rupiah dan mengembalikan boolean: true jika pengembalian dana berhasil dan false jika gagal. Dengan begitu, class yang mengimplementasikan RefundablePayment, seperti EWalletPayment, wajib menyediakan isi dari ketiga method tersebut: getName(), pay(), dan refund(). Interface ini juga memisahkan metode pembayaran yang mendukung pengembalian dana dari yang tidak. BankTransferPayment hanya mengimplementasikan PaymentMethod, sehingga tidak bisa dikirim ke CheckoutService.refund() yang parameternya bertipe RefundablePayment, dan kesalahan tersebut sudah terdeteksi saat kompilasi. Selain itu, karena RefundablePayment adalah turunan dari PaymentMethod, objeknya tetap bisa dipakai di mana pun yang membutuhkan PaymentMethod, misalnya pada CheckoutService.checkout(), sehingga ini merupakan penerapan polimorfisme.

- Interface Installment Payment

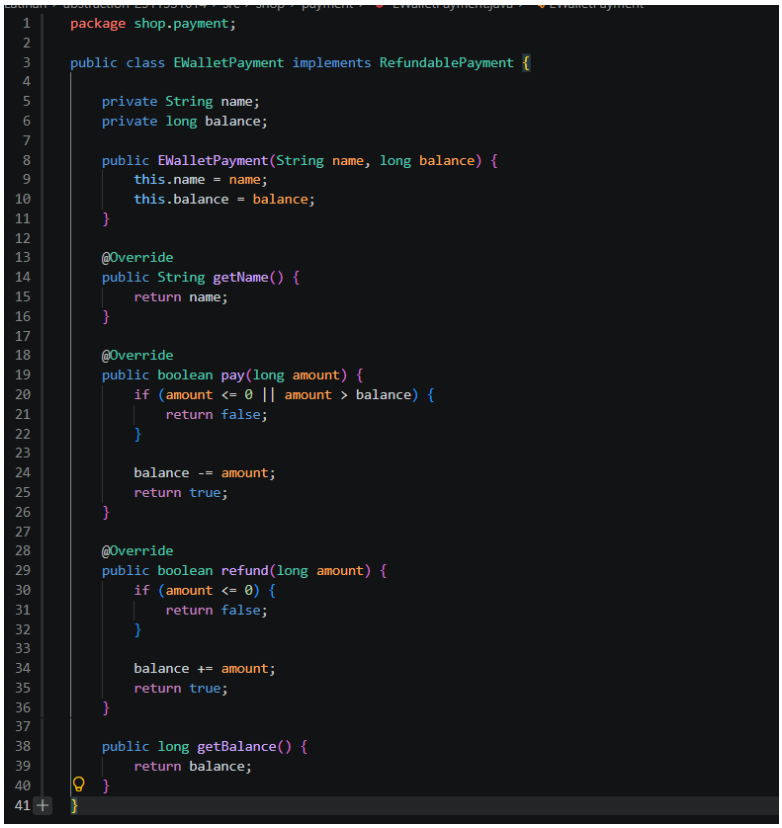


Kode ini mendefinisikan InstallmentPayment di package shop.payment sebagai interface yang memperluas PaymentMethod lewat kata kunci extends, sehingga aturan getName() dan pay(long amount) otomatis ikut diwarisi. Interface ini merepresentasikan kemampuan pembayaran secara cicilan. Di dalamnya ada dua method.

Pertama, maxInstallmentMonths() adalah method abstract tanpa isi, sehingga setiap class yang mengimplementasikannya wajib menentukan batas maksimal jumlah bulan cicilannya sendiri. Kedua, monthlyInstallment(long amount, int months) adalah default method (ditandai kata kunci default), yaitu method yang sudah memiliki isi sehingga langsung diwarisi oleh class pengimplementasi tanpa harus ditulis ulang. Method ini menghitung besar cicilan per bulan dengan dua langkah. Pertama, dilakukan validasi: jika months kurang dari 1 atau melebihi maxInstallmentMonths(), dilempar IllegalArgumentException dengan pesan yang menyebutkan batas bulan yang diperbolehkan. Kedua, jika valid, cicilan dihitung dengan rumus (amount + months - 1) / months, yaitu pembagian bilangan bulat yang dibulatkan ke atas. Sebagai contoh, Rp100.000 dibagi 3 bulan menghasilkan Rp33.334 per bulan (bukan Rp33.333), sehingga total cicilan tidak kurang dari jumlah yang harus dibayar. Karena maxInstallmentMonths() dipanggil di dalam default method tetapi isinya ditentukan oleh class pengimplementasi, perilaku validasinya menyesuaikan tiap class, dan ini merupakan penerapan polimorfisme.

d. Penambahan dan perubahan pada class berikut.

- Class EWalletPaymet



```java
package shop.payment;

public class EWalletPayment implements RefundablePayment {

    private String name;
    private long balance;

    public EWalletPayment(String name, long balance) {
        this.name = name;
        this.balance = balance;
    }

    @Override
    public String getName() {
        return name;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }

        balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }

        balance += amount;
        return true;
    }

    public long getBalance() {
        return balance;
    }
}
```

Kode ini mendefinisikan class EWalletPayment di package shop.payment yang mengimplementasikan interface RefundablePayment (lewat implements), yaitu metode pembayaran berupa dompet digital yang saldonya bisa dipakai membayar dan dikembalikan (refund). Class ini memiliki dua atribut private, yaitu name (nama e-wallet) dan balance (saldo dalam rupiah, bertipe long), sehingga tidak bisa diakses langsung dari luar class. Berbeda dengan class sebelumnya, atribut ini tidak bersifat final karena nilai saldo memang harus berubah setiap kali terjadi pembayaran atau refund. Constructor-nya bersifat public, menerima name dan balance, lalu menyimpannya ke atribut melalui this. Method getName() yang diberi anotasi @Override merupakan implementasi dari interface dan mengembalikan nama e-wallet. Method pay() juga di-@Override dan memproses pembayaran: jika amount kurang dari atau sama dengan 0, atau lebih besar dari saldo, pembayaran ditolak dengan

mengembalikan false. Jika valid, saldo dikurangi sebesar amount dan method mengembalikan true. Method refund() mengembalikan dana ke saldo: jika amount kurang dari atau sama dengan 0, dikembalikan false, sedangkan jika valid maka saldo ditambah sebesar amount dan dikembalikan true. Terakhir, getBalance() adalah getter untuk membaca saldo saat ini. Karena RefundablePayment kemungkinan merupakan turunan dari PaymentMethod, class ini wajib mengimplementasikan getName(), pay(), dan refund(), sehingga dapat dipakai sebagai metode pembayaran biasa maupun yang mendukung pengembalian dana.

- Class CreditCardPayment

```java
package shop.payment;

public class CreditCardPayment implements RefundablePayment, InstallmentPayment {

    private String cardHolder;
    private long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long creditLimit) {
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;
    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount > creditLimit) {
            return false;
        }

        usedCredit += amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }

        usedCredit -= amount;
        return true;
    }

    @Override
    public int maxInstallmentMonths() {
        return 12;
    }

    public long getUsedCredit() {
        return usedCredit;
    }
}
```

Kode ini mendefinisikan class CreditCardPayment di package shop.payment yang mengimplementasikan dua interface sekaligus, yaitu RefundablePayment dan InstallmentPayment (dipisahkan koma setelah implements), sehingga kartu kredit ini dapat dipakai untuk membayar, menerima pengembalian dana, dan membayar secara

cicilan. Class ini memiliki tiga atribut private, yaitu cardHolder (nama pemegang kartu), creditLimit (batas kredit), dan usedCredit (kredit yang sudah terpakai), keduanya bertipe long untuk nominal rupiah, sehingga tidak bisa diakses langsung dari luar class. Atribut ini tidak bersifat final karena usedCredit harus berubah setiap kali terjadi pembayaran atau refund. Constructor-nya bersifat public, menerima cardHolder dan creditLimit lalu menyimpannya melalui this, sedangkan usedCredit langsung diisi 0 karena kartu baru belum dipakai. Method getName() yang diberi anotasi @Override mengembalikan teks gabungan "Kartu Kredit (" dengan nama pemegang kartu, misalnya "Kartu Kredit (Siti Rahma)". Method pay() memproses pembayaran: jika amount kurang dari atau sama dengan 0, atau jika kredit terpakai ditambah amount melebihi creditLimit, pembayaran ditolak dengan mengembalikan false. Jika valid, usedCredit ditambah sebesar amount dan method mengembalikan true. Berbeda dengan e-wallet yang mengurangi saldo, kartu kredit menambah jumlah kredit terpakai hingga batas limit. Method refund() mengembalikan dana: jika amount kurang dari atau sama dengan 0, atau lebih besar dari usedCredit, ditolak dengan false, sedangkan jika valid maka usedCredit dikurangi sebesar amount dan dikembalikan true. Method maxInstallmentMonths() merupakan implementasi dari method abstract di InstallmentPayment dan mengembalikan 12, artinya cicilan maksimal 12 bulan, sehingga default method monthlyInstallment() akan menolak jumlah bulan di luar rentang 1 sampai 12. Terakhir, getUsedCredit() adalah getter untuk membaca kredit yang sudah terpakai. Karena mengimplementasikan dua interface (multiple interfaces) yang sama-sama turunan PaymentMethod, objek class ini dapat diperlakukan sebagai PaymentMethod, RefundablePayment, maupun InstallmentPayment, sehingga ini merupakan penerapan polimorfisme.

- Class Checkout Service



```java
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) { // a capability, not a class
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items, ShippingService shipping) {
        return subtotal(items) + shipping.calculateCost(totalWeight(items));
    }

    public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
        long totalAmount = total(items, shipping);
        return payment.pay(totalAmount);
    }

    public boolean refund(RefundablePayment payment, long amount) {
    return payment.refund(amount);
    }
}
```

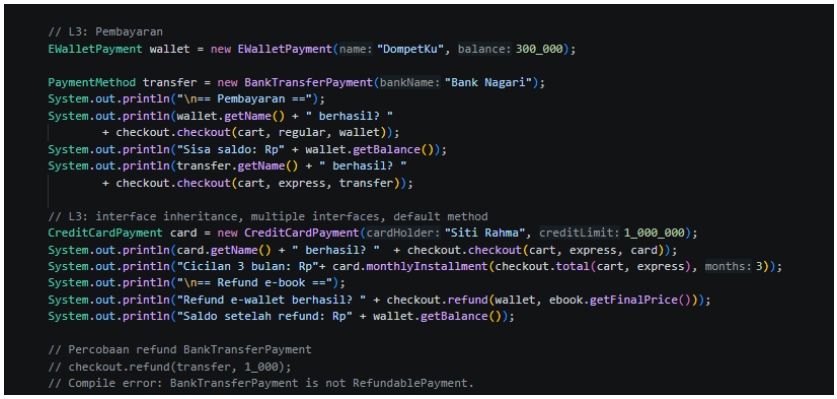
Penambahan kode pada bagian sini:



```java
    public boolean refund(RefundablePayment payment, long amount) {
    return payment.refund(amount);
    }
}
```

Kode ini adalah method refund() di dalam class CheckoutService yang bertugas memproses pengembalian dana. Method ini bersifat public dan mengembalikan nilai boolean, yaitu true jika pengembalian dana berhasil dan false jika gagal. Parameternya ada dua: payment bertipe RefundablePayment, yaitu metode pembayaran yang mendukung pengembalian dana, dan amount bertipe long, yaitu jumlah dana dalam rupiah yang akan dikembalikan. Isinya hanya satu baris, yaitu return payment.refund(amount), artinya method ini meneruskan permintaan ke method refund() milik objek payment dan langsung mengembalikan hasilnya. Cara pengembalian dana yang sebenarnya ditentukan oleh class pembayaran masing-masing: pada EWalletPayment saldo ditambah, sedangkan pada CreditCardPayment kredit terpakai dikurangi. Karena parameternya bertipe interface RefundablePayment, method ini bisa dipakai oleh semua metode pembayaran yang mengimplementasikannya tanpa perlu tahu jenis konkretnya, sehingga ini merupakan penerapan polimorfisme.

e. Legkapi ShopApp denhan pembayaran



```
// L3: Pembayaran
EWalletPayment wallet = new EWalletPayment(name: "DompetKu", balance: 300_000);

PaymentMethod transfer = new BankTransferPayment(bankName: "Bank Nagari");
System.out.println("\n== Pembayaran ==");
System.out.println(wallet.getName() + " berhasil? "
        + checkout.checkout(cart, regular, wallet));
System.out.println("Sisa saldo: Rp" + wallet.getBalance());
System.out.println(transfer.getName() + " berhasil? "
        + checkout.checkout(cart, express, transfer));

// L3: interface inheritance, multiple interfaces, default method
CreditCardPayment card = new CreditCardPayment(cardHolder: "Siti Rahma", creditLimit: 1_000_000);
System.out.println(card.getName() + " berhasil? "  + checkout.checkout(cart, express, card));
System.out.println("Cicilan 3 bulan: Rp"+ card.monthlyInstallment(checkout.total(cart, express), months: 3));
System.out.println("\n== Refund e-book ==");
System.out.println("Refund e-wallet berhasil? " + checkout.refund(wallet, ebook.getFinalPrice()));
System.out.println("Saldo setelah refund: Rp" + wallet.getBalance());

// Percobaan refund BankTransferPayment
// checkout.refund(transfer, 1_000);
// Compile error: BankTransferPayment is not RefundablePayment.
```

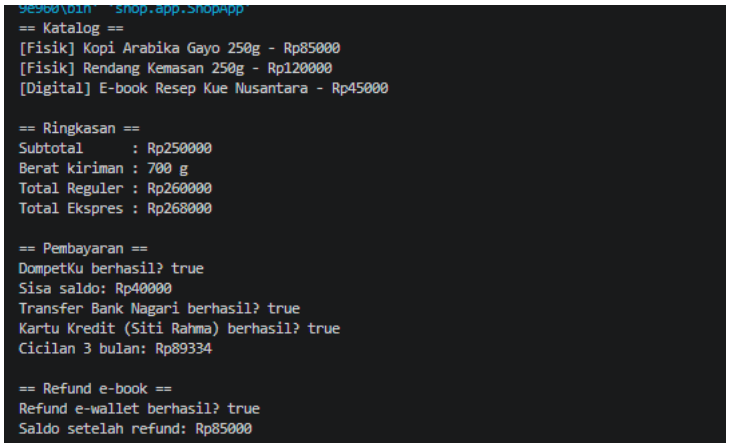
Bagian kode ini adalah lanjutan method main pada ShopApp yang menjalankan simulasi pembayaran, cicilan, dan refund. Pertama dibuat objek wallet bertipe EWalletPayment bernama “DompetKu” dengan saldo Rp300.000, dan transfer bertipe PaymentMethod yang berisi BankTransferPayment “Bank Nagari”. Pembayaran dengan wallet memakai pengiriman reguler sehingga totalnya Rp260.000, hasilnya true, dan saldo tersisa Rp40.000 yang dicetak lewat getBalance(). Pembayaran dengan transfer memakai pengiriman ekspres dan berhasil karena BankTransferPayment hanya mensimulasikan pembayaran selama jumlahnya positif.

Pada bagian L3, dibuat objek card bertipe CreditCardPayment atas nama “Siti Rahma” dengan batas kredit Rp1.000.000. Pembayaran keranjang dengan pengiriman ekspres sebesar Rp268.000 berhasil karena masih di bawah batas kredit, sehingga usedCredit menjadi Rp268.000. Baris berikutnya mencetak cicilan 3 bulan lewat card.monthlyInstallment(checkout.total(cart, express), 3). Method ini adalah default method dari InstallmentPayment, sehingga bisa langsung dipanggil pada card walaupun tidak ditulis ulang di class-nya. Hasilnya Rp89.334 per bulan, karena (268.000 + 3 - 1) / 3 dibulatkan ke atas. Bagian ini menunjukkan multiple interfaces dan default method, yaitu satu class yang memiliki kemampuan refund dan cicilan sekaligus.

Pada bagian “Refund e-book”, checkout.refund(wallet, ebook.getFinalPrice()) mengembalikan dana sebesar harga akhir e-book ke wallet. Karena DigitalProduct mendapat diskon 10%,

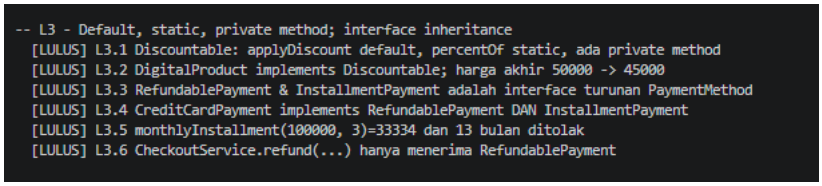
nilainya Rp45.000 (bukan Rp50.000), sehingga saldo naik dari Rp40.000 menjadi Rp85.000 dan hasilnya true. Kode ini lolos kompilasi karena wallet adalah EWalletPayment yang mengimplementasikan RefundablePayment, lalu baris "Saldo setelah refund" mencetak Rp85.000. Terakhir, baris checkout.refund(transfer, 1_000) dijadikan komentar karena jika diaktifkan akan menimbulkan compile error: BankTransferPayment hanya mengimplementasikan PaymentMethod, bukan RefundablePayment.

Output:



f. Selfcheck

Setelah melakukan semua Langkah diatas lakukan uji coba di bagian selfcheck melihat dibagian mana saja yang gagal atau lulus. Output yangg di hasil kan saat menjalankan selfcheck:



Semua Latihan 3 berhasil di jalankan atau lulus.

4) Latihan 4(Final method dan final class)

a. Kunci rumus ongkir (Class ShippingService)

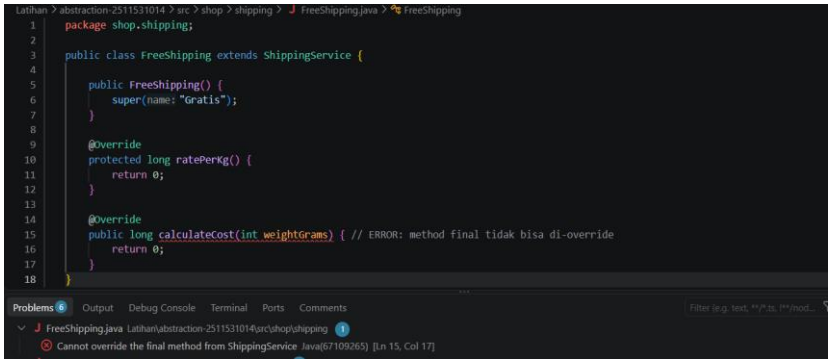


```
/** Weight is rounded up: 700 g -> 1 Kg */
public final long calculateCost(int weightGrams) {
    if (weightGrams <= 0) {
        return 0;
    }

    long kilograms = (weightGrams + 999L) / 1000;
    return kilograms * ratePerKg();
}
}
```

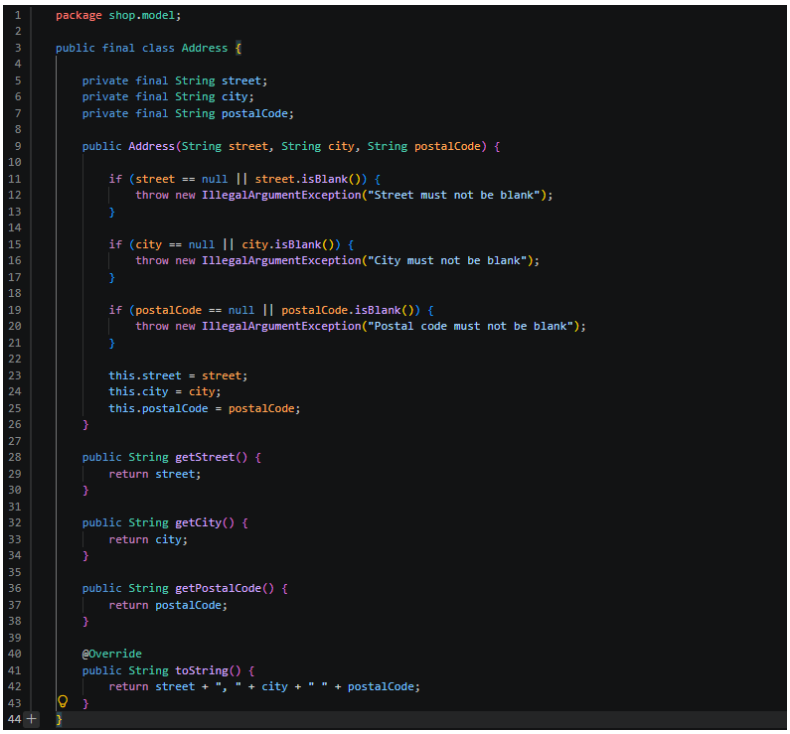
Ini adalah tambahan dari class Shipping service yang sudah dibahas di bagian latihan 1. Kode ini adalah method calculateCost() di dalam class ShippingService yang bertugas menghitung biaya pengiriman berdasarkan berat barang. Method ini bersifat public final, artinya bisa dipanggil dari luar class tetapi tidak bisa di-override oleh subclass, sehingga cara perhitungan biayanya selalu seragam untuk semua layanan pengiriman. Method ini menerima parameter weightGrams bertipe int (berat dalam gram) dan mengembalikan nilai long berupa biaya dalam rupiah. Komentar Javadoc di atasnya menjelaskan bahwa berat dibulatkan ke atas, misalnya 700 gram dihitung sebagai 1 kg. Pada langkah pertama, jika weightGrams kurang dari atau sama dengan 0, method langsung mengembalikan 0 karena tidak ada barang yang perlu dikirim. Jika berat valid, berat dikonversi ke kilogram dengan rumus (weightGrams + 999L) / 1000. Penambahan 999 membuat pembagian bilangan bulat selalu membulatkan ke atas, misalnya 700 gram menjadi 1 kg dan 1.001 gram menjadi 2 kg, sedangkan huruf L membuat angka 999 bertipe long agar hasilnya sesuai dengan variabel kilograms. Terakhir, jumlah kilogram dikalikan dengan ratePerKg() untuk menghasilkan total biaya. Method ratePerKg() bersifat abstract dan diimplementasikan oleh subclass, sehingga tarifnya berbeda tergantung layanan yang dipakai, misalnya Rp10.000 per kg pada RegularShipping dan Rp18.000 per kg pada ExpressShipping. Pola ini disebut template method, karena langkah perhitungan sudah ditetapkan di class induk sementara tarifnya ditentukan oleh subclass.

Uji coba dnegan class FreeShipping



Class FreeShipping dibuat sebagai subclass dari ShippingService dengan nama layanan “Gratis” yang diteruskan lewat super("Gratis"). Method ratePerKg() di-override dengan mengembalikan 0, dan ini diperbolehkan karena method tersebut abstract sehingga memang wajib diimplementasikan. Namun, percobaan meng-override calculateCost() menghasilkan error kompilasi “Cannot override the final method from ShippingService”, karena method tersebut dideklarasikan final di class induk. Hal ini membuktikan bahwa final pada method mencegah subclass mengubah cara perhitungan biaya, sehingga algoritma tetap seragam (template method). Untuk membuat pengiriman gratis, cukup dengan membiarkan calculateCost() apa adanya dan mengembalikan 0 pada ratePerKg(), karena biaya yang dihasilkan otomatis menjadi 0.

b. Class Address



```java
package shop.model;

public final class Address {

    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city, String postalCode) {

        if (street == null || street.isBlank()) {
            throw new IllegalArgumentException("Street must not be blank");
        }

        if (city == null || city.isBlank()) {
            throw new IllegalArgumentException("City must not be blank");
        }

        if (postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Postal code must not be blank");
        }

        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getStreet() {
        return street;
    }

    public String getCity() {
        return city;
    }

    public String getPostalCode() {
        return postalCode;
    }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

Kode ini mendefinisikan class Address di package shop.model sebagai class final, artinya class ini tidak bisa diwariskan oleh class lain sehingga strukturnya terjamin tidak diubah lewat pewarisan. Class ini merepresentasikan alamat pengiriman dan memiliki tiga atribut private final, yaitu street (jalan), city (kota), dan postalCode (kode pos), sehingga tidak bisa diakses langsung dari luar class dan nilainya tidak bisa diubah setelah objek dibuat (immutable). Constructor-nya bersifat public dan menerima ketiga data tersebut, lalu memvalidasinya satu per satu: jika street, city, atau postalCode bernilai null atau kosong, akan dilempar IllegalArgumentException dengan pesan “Street must not be blank”, “City must not be blank”, atau “Postal code must not be blank”. Setelah lolos validasi, nilai parameter disimpan ke atribut melalui this, sehingga objek Address yang terbentuk dijamin selalu berisi data yang valid. Method getStreet(), getCity(), dan getPostalCode() dibuat public sebagai getter agar data yang bersifat private tetap bisa dibaca dari luar class. Terakhir, method toString() di-@Override untuk menggantikan versi

bawaan milik class Object, dengan mengembalikan teks gabungan street, city postalCode, misalnya “Jl. Moh Hatta No. 10, Padang 25121”, sehingga objek Address dapat langsung dicetak dalam bentuk teks yang mudah dibaca, misalnya pada struk pesanan.

c. Selfcheck

Setelah melakukan semua Langkah diatas lakukan uji coba di bagian selfcheck melihat dibagian mana saja yang gagal atau lulus. Output yangg di hasil kan saat menjalankan selfcheck:

```
-- L4 - final method & final class
  [LULUS] L4.1 ShippingService.calculateCost(int) adalah final method
  [LULUS] L4.2 Address final class, semua field private final, tanpa setter
```

Semua Latihan 4 berhasil di jalankan atau lulus.

5) Latihan 5 (Nested Class)

a. Class Order

```java
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress) {
        if (orderId == null || orderId.isBlank() || shippingAddress == null) {
            throw new IllegalArgumentException("Order id and address are required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() {
        return orderId;
    }
    public Address getShippingAddress() {
        return shippingAddress;
    }
    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }
    public List<Line> getLines() {
        return Collections.unmodifiableList(lines);
    }
}
```

Kode ini mendefinisikan class Order di package shop.model yang merepresentasikan satu pesanan pelanggan, berisi kode pesanan, alamat pengiriman, dan daftar barang yang dipesan. Di bagian atas, ArrayList, Collections, dan List diimpor dari java.util untuk mengelola daftar barang. Class ini memiliki tiga atribut private final, yaitu orderId (kode pesanan), shippingAddress (alamat pengiriman bertipe Address), dan lines (daftar baris pesanan bertipe List<Line>

yang langsung diisi new ArrayList<>()). Karena bersifat private final, atribut ini tidak bisa diakses langsung dari luar dan referensinya tidak bisa diganti setelah objek dibuat. Pemakaian Address sebagai atribut menunjukkan hubungan komposisi, yaitu sebuah Order memiliki sebuah Address.

Constructor-nya bersifat public dan menerima orderId serta shippingAddress. Di dalamnya dilakukan validasi: jika orderId bernilai null atau kosong, atau shippingAddress bernilai null, akan dilempar IllegalArgumentException dengan pesan "Order id and address are required". Setelah lolos validasi, nilai parameter disimpan ke atribut melalui this. Method getOrderId() dan getShippingAddress() adalah getter untuk membaca kode pesanan dan alamat pengiriman. Method addLine(Product product, int quantity) menambahkan barang ke pesanan dengan membuat objek Line baru dari produk dan jumlahnya, lalu memasukkannya ke list lines. Parameter bertipe Product sehingga produk fisik maupun digital bisa dimasukkan, yang merupakan penerapan polimorfisme. Terakhir, getLines() mengembalikan daftar baris pesanan dalam bentuk Collections.unmodifiableList(lines), yaitu tampilan hanya-baca (read-only). Dengan begitu, kode di luar class bisa melihat isi pesanan tetapi tidak bisa menambah atau menghapus barang secara langsung, dan penambahan hanya bisa lewat addLine(). Ini menjaga enkapsulasi, karena final pada lines hanya mengunci referensinya, bukan isi list-nya.

b. Class Line dan Class Receipt dan penambahan di Class Order

- Class Line



```java
// Class Line
public static class Line {

    private final Product product;
    private final int quantity;

    public Line(Product product, int quantity) {
        if (product == null || quantity <= 0) {
            throw new IllegalArgumentException(
                    "Product and quantity must be valid");
        }
        this.product = product;
        this.quantity = quantity;
    }
    public Product getProduct() {
        return product;
    }
    public int getQuantity() {
        return quantity;
    }
    public long subtotal() {
        return product.getFinalPrice() * quantity;
    }
}
```

Kode ini mendefinisikan class Line sebagai public static class di dalam class Order, yaitu static nested class yang merepresentasikan satu baris pesanan (satu produk beserta jumlahnya). Modifier static membuat Line tidak terikat pada objek Order tertentu, sehingga bisa dibuat tanpa harus ada objek Order terlebih dahulu, dan dipanggil dari luar dengan nama Order.Line. Class ini memiliki dua atribut private final, yaitu product (produk yang dipesan, bertipe Product) dan quantity (jumlah yang dipesan), sehingga tidak bisa diakses langsung dari luar dan nilainya tidak bisa diubah setelah objek dibuat. Constructor-nya bersifat public dan menerima product serta quantity, lalu memvalidasinya: jika product bernilai null atau quantity kurang dari atau sama dengan 0, akan dilempar IllegalArgumentException dengan pesan “Product and quantity must be valid”. Setelah lolos validasi, nilai parameter disimpan ke atribut melalui this. Method getProduct() dan getQuantity() adalah getter untuk membaca produk dan jumlahnya. Terakhir, method subtotal() menghitung total harga baris tersebut dengan mengalikan product.getFinalPrice() dengan quantity. Karena memakai getFinalPrice(), hasilnya mengikuti harga akhir tiap jenis produk, misalnya e-book yang sudah terdiskon 10%,

sehingga ini merupakan penerapan polimorfisme. Sebagai contoh, 2 kopi seharga Rp85.000 menghasilkan subtotal Rp170.000.

- Method getItemsTotal()

```
// Menghitung total harga semua produk
public long getItemsTotal() {
    long total = 0;
    for (Line line : lines) {
        total += line.subtotal();
    }
    return total;
}
```

Kode ini adalah method getItemsTotal() di dalam class Order yang bertugas menghitung total harga seluruh barang dalam pesanan. Method ini bersifat public dan mengembalikan nilai long karena nominalnya dalam rupiah. Di dalamnya dibuat variabel total yang diawali dengan 0 sebagai penampung hasil penjumlahan. Perulangan for-each kemudian menelusuri setiap objek Line pada list lines, dan pada tiap putaran total ditambah dengan hasil line.subtotal(), yaitu harga akhir produk dikalikan jumlahnya. Karena subtotal() memakai getFinalPrice(), total yang dihasilkan sudah memperhitungkan diskon pada produk yang memilikinya, misalnya e-book yang terdiskon 10%, sehingga ini merupakan penerapan polimorfisme. Setelah semua baris dijumlahkan, nilai total dikembalikan lewat return. Sebagai contoh, pada ShopApp pesanan berisi 2 kopi (Rp170.000) dan 1 e-book (Rp45.000), sehingga hasilnya Rp215.000.

- Class Receipt

```
// Class Receipt (inner class, tanpa static)
public class Receipt {
    public String toText() {
        StringBuilder text = new StringBuilder();
        text.append("STRUK PESANAN ")
                .append(Order.this.orderId)
                .append("\n");
        for (Line line : lines) {
            text.append("- ")
                    .append(line.getProduct().getName())
                    .append(" x")
                    .append(line.getQuantity())
                    .append(" = Rp")
                    .append(line.subtotal())
                    .append("\n");
        }
        text.append("Total pesanan: Rp")
                .append(getItemsTotal())
                .append("\n");
        text.append("Kirim ke: ")
                .append(Order.this.shippingAddress);
        return text.toString();
    }
}
}
```

Kode ini mendefinisikan class Receipt sebagai public class tanpa static di dalam class Order, yaitu inner class (non-static nested class) yang bertugas menyusun struk pesanan dalam bentuk teks. Karena tidak bersifat static, setiap objek Receipt terikat pada satu objek Order tertentu dan tidak bisa dibuat tanpa objek Order, sehingga pembuatannya memakai sintaks order.new Receipt() seperti di ShopApp. Keterikatan ini membuat Receipt dapat langsung mengakses atribut lines dan orderId milik Order, termasuk yang bersifat private, serta memanggil method getItemsTotal(). Class ini hanya memiliki satu method, yaitu toText(), yang mengembalikan String.

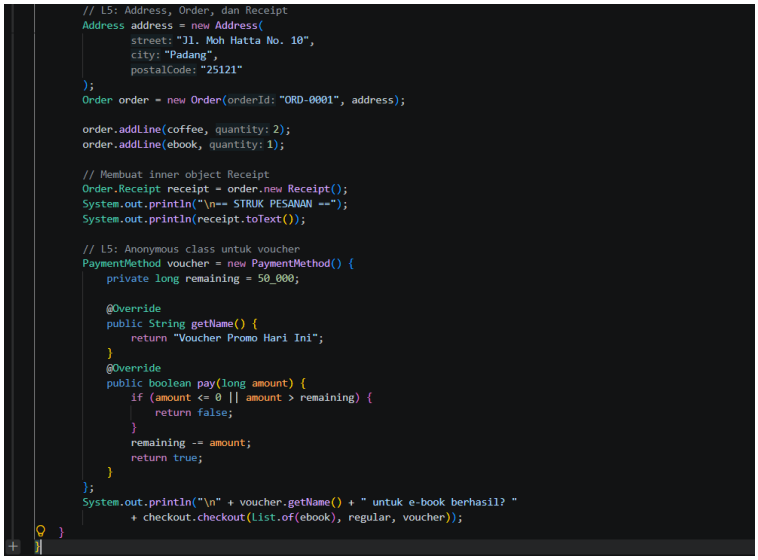
Di dalam toText(), dibuat objek StringBuilder bernama text untuk menyusun teks secara bertahap, karena lebih efisien daripada menggabungkan String berulang kali dengan +. Baris pertama diisi "STRUK PESANAN " diikuti kode pesanan lewat Order.this.orderId, yaitu sintaks untuk merujuk ke objek Order pembungkus dari dalam inner class, lalu ditutup dengan karakter baris baru \n. Perulangan for-each kemudian menelusuri setiap Line pada list lines, dan untuk tiap baris ditambahkan teks berformat - nama produk x jumlah = Rp subtotal, yang diambil dari getProduct().getName(), getQuantity(), dan subtotal(). Setelah semua baris, ditambahkan baris "Total pesanan: Rp" diikuti hasil getItemsTotal(), lalu baris "Kirim ke: " diikuti Order.this.shippingAddress. Objek Address tersebut otomatis dipanggil toString()-nya sehingga tampil sebagai alamat lengkap. Terakhir, text.toString() dikembalikan sebagai hasil berupa satu String utuh.

- Method createReceipt()

```
// Membuat receipt
public Receipt createReceipt() {
    return new Receipt();
}
```

Kode ini adalah method createReceipt() di dalam class Order yang berfungsi sebagai factory method, yaitu method yang tugasnya membuat dan mengembalikan objek Receipt untuk pesanan tersebut. Method ini bersifat public dan mengembalikan objek bertipe Receipt, yaitu inner class milik Order. Isinya hanya satu baris, return new Receipt();. Karena Receipt adalah inner class non-static, objek yang dibuat dari dalam Order otomatis terikat pada objek Order yang memanggil method ini (this), sehingga tidak perlu menulis this.new Receipt() secara eksplisit. Dengan adanya method ini, kode di luar class cukup memanggil order.createReceipt() tanpa harus memakai sintaks order.new Receipt() yang lebih rumit seperti di ShopApp, dan detail pembuatan struk tetap tersembunyi di dalam Order, sehingga ini juga menjaga enkapsulasi.

c. Lengkapi ShopApp



```
        // L5: Address, Order, dan Receipt
        Address address = new Address(
                street: "Jl. Moh Hatta No. 10",
                city: "Padang",
                postalCode: "25121"
        );
        Order order = new Order(orderId: "ORD-0001", address);

        order.addLine(coffee, quantity: 2);
        order.addLine(ebook, quantity: 1);

        // Membuat inner object Receipt
        Order.Receipt receipt = order.new Receipt();
        System.out.println("\n== STRUK PESANAN ==");
        System.out.println(receipt.toText());

        // L5: Anonymous class untuk voucher
        PaymentMethod voucher = new PaymentMethod() {
            private long remaining = 50_000;

            @Override
            public String getName() {
                return "Voucher Promo Hari Ini";
            }
            @Override
            public boolean pay(long amount) {
                if (amount <= 0 || amount > remaining) {
                    return false;
                }
                remaining -= amount;
                return true;
            }
        };
        System.out.println("\n" + voucher.getName() + " untuk e-book berhasil? "
                + checkout.checkout(List.of(ebook), regular, voucher));
    }
}
```

Bagian kode ini adalah lanjutan dan penutup method main pada ShopApp yang mencakup pembuatan alamat, pesanan, struk, serta pembayaran dengan voucher. Pertama dibuat objek address bertipe Address dengan jalan “Jl. Moh Hatta No. 10”, kota “Padang”, dan kode pos “25121”. Objek ini lalu dipakai untuk membuat order bertipe Order dengan kode “ORD-0001”. Barang ditambahkan lewat addLine(), yaitu 2 buah coffee dan 1 buah ebook, sehingga pesanan berisi dua baris Line.

Selanjutnya dibuat objek receipt bertipe Order.Receipt dengan sintaks order.new Receipt(). Sintaks ini diperlukan karena Receipt adalah inner class non-static, sehingga setiap objeknya harus terikat pada objek Order tertentu, dalam hal ini order. Method receipt.toText() kemudian dipanggil dan hasilnya dicetak, berisi kode pesanan, daftar barang beserta subtotalnya, total pesanan Rp215.000, dan alamat pengiriman.

Pada bagian terakhir, voucher dibuat sebagai anonymous class, yaitu class tanpa nama yang langsung mengimplementasikan interface PaymentMethod lewat new PaymentMethod() { ... } sehingga tidak perlu membuat file class baru. Class ini memiliki atribut remaining bernilai Rp50.000 sebagai sisa nilai voucher. Method getName() mengembalikan "Voucher Promo Hari Ini", sedangkan pay() menolak pembayaran (false) jika amount kurang dari atau sama dengan 0 atau melebihi remaining, dan jika valid maka remaining dikurangi sebesar amount lalu dikembalikan true. Voucher ini dipakai pada checkout.checkout(List.of(ebook), regular, voucher) untuk membeli e-book saja dengan pengiriman reguler. Karena e-book tidak punya berat, ongkir bernilai 0, sehingga total hanya Rp45.000, masih di bawah sisa voucher Rp50.000, dan pembayaran berhasil (true).

d. SelfCheck

Setelah melakukan semua Langkah diatas lakukan uji coba di bagian selfcheck melihat dibagian mana saja yang gagal atau lulus. Output yangg di hasil kan saat menjalankan selfcheck:

```
-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan PaymentMethod

=== HASIL: 23 / 23 pemeriksaan lulus ===
PS C:\Users\naysh\OneDrive\TUGAS SMS 3\PRK PBO\praktikum6>
```

Semua Latihan 5 berhasil di jalankan atau lulus. Dan Hasil 23/23 Pemeriksaan lulus.

2. Tugas Individu

1) Tugas 0 Parameter pribadi berdasarkan NIM

Parameter tugas untuk NIM 2511531014
CARD_ADMIN_PERCENT = 2
TRANSFER_ADMIN_FEE = 4500
SMS_MAX_LENGTH = 114

```
Problems 9  Output  Debug Console  Terminal  Ports  Comments

PS C:\Users\naysh\OneDrive\TUGAS SMS 3\PRK PBO\praktikum6> cd Latihan\abstraction-2511531014
>> javac -d bin src\tools\NimParams.java
>> java -cp bin tools.NimParams 2511531014
Parameter tugas untuk NIM 2511531014
  CARD_ADMIN_PERCENT  = 2
  TRANSFER_ADMIN_FEE  = 4500
  SMS_MAX_LENGTH      = 114
PS C:\Users\naysh\OneDrive\TUGAS SMS 3\PRK PBO\praktikum6\Latihan\abstraction-2511531014>
```

Latihan > abstraction-2511531014 > src > shop > J StudentInfo.java > ...

```java
package shop;

/**
 * Identity and personal parameters for the take-home assignment (Tugas).
 * 1. Write your NIM and name.
 * 2. Run: java -cp out tools.NimParams <your NIM>
 * 3. Copy the three printed values below.
 */
public final class StudentInfo {
    public static final String NIM = "2511531014";   // TODO: e.g. "2401010057"
    public static final String NAME = "Naysha Aprilia Rizki";  // TODO: your full name

    public static final int CARD_ADMIN_PERCENT = 2;   // TODO: from tools.NimParams
    public static final long TRANSFER_ADMIN_FEE = 4500;  // TODO: from tools.NimParams
    public static final int SMS_MAX_LENGTH = 114;       // TODO: from tools.NimParams

    private StudentInfo() {
        // constants only; no objects
    }
}
```

2) Tugas 1 Evolusi API dan konflik diamond

a. PaymentMethod.adminFee(long) adalah default method; e-wallet = 0

- Cara yang salah

Tambah kan method abstract long adminFee(long amount); ke PaymentMethod

ihan > abstraction-2511531014 > src > shop > payment > J PaymentMethod.java > ...

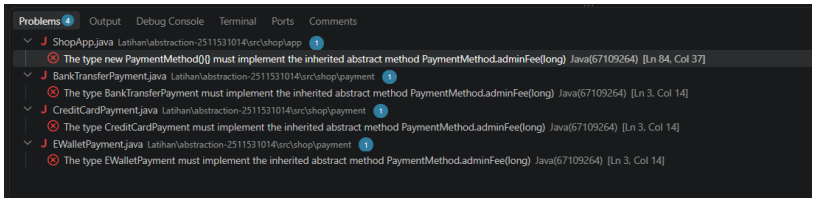
```java
package shop.payment;

/** Contract for every payment method */
public interface PaymentMethod {
    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);

    long adminFee(long amount);
}
```

hasil nya akan error:



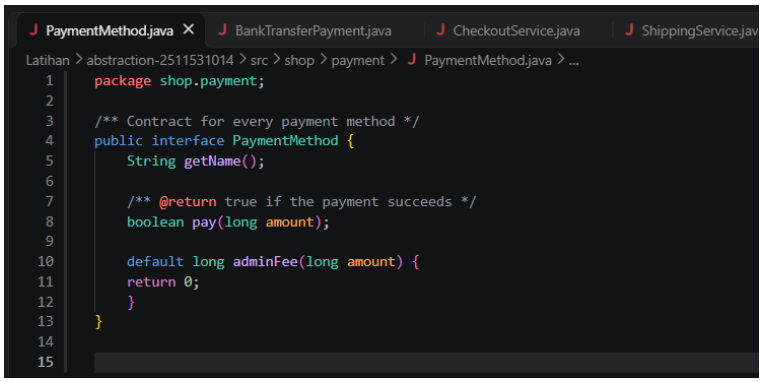
1. ada berapa error, class mana saja, dan mengapa anonymous class voucher di ShopApp juga ikut rusak?

Jawab:

Ada 4 class yang error yaitu ShopApp.java(bagian anonymous class), BankTransferPayment, CreditCardPayment, EWalletPayment.

Anonymous class tetap sebuah class konkret yang meng-implement PaymentMethod, hanya saja tanpa nama. Aturan Java berlaku sama untuknya: class konkret wajib mengimplementasikan semua abstract method dari interface yang dipakainya. Voucher di ShopApp hanya meng-override getName() dan pay(), jadi begitu adminFee() ditambahkan sebagai abstract method, voucher tidak lagi memenuhi kontrak dan compiler menolaknya.

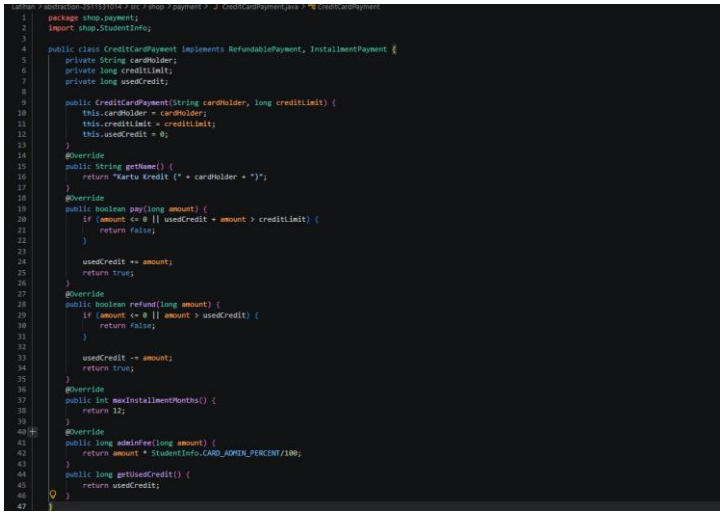
- Default Method



Kode ini adalah interface PaymentMethod yang berfungsi sebagai kontrak untuk semua metode pembayaran. Interface ini punya dua method abstract, yaitu getName() dan pay(long amount), yang wajib diisi oleh setiap class yang meng-implement-nya. Selain itu ada method default adminFee(long amount) yang langsung berisi return 0, sehingga secara bawaan tidak ada biaya admin. Class seperti EWalletPayment dan anonymous class voucher tidak perlu diubah karena otomatis memakai versi default ini, sedangkan CreditCardPayment dan BankTransferPayment bisa meng-override-nya. Dengan default method, interface dapat ditambah kemampuan baru tanpa merusak kode lama, berbeda dengan abstract method yang menyebabkan 4 error pada T1.a.

b. Biaya admin kartu kredit & transfer sesuai parameter NIM
Override sesuai kebutuhan

- CreditCardPayment



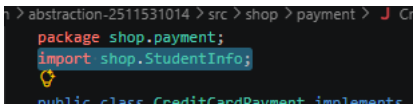
```
package shop.payment;
import shop.StudentInfo;

public class CreditCardPayment implements RefundablePayment, InstallmentPayment {
    private String cardHolder;
    private long creditLimit;
    private long usedCredit;

    public CreditCardPayment(String cardHolder, long creditLimit) {
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
        this.usedCredit = 0;
    }
    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }
    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount > creditLimit) {
            return false;
        }

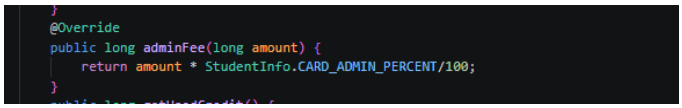
        usedCredit += amount;
        return true;
    }
    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }

        usedCredit -= amount;
        return true;
    }
    @Override
    public int maxInstallmentMonths() {
        return 12;
    }
    @Override
    public long adminFee(long amount) {
        return amount * StudentInfo.CARD_ADMIN_PERCENT/100;
    }
    public long getUsedCredit() {
        return usedCredit;
    }
}
```



```
> abstraction-2511531014 > src > shop > payment > J C
package shop.payment;
import shop.StudentInfo;
```

Di lakukan penambahan dibagian import shop.StudentInfo dipakai supaya CreditCardPayment bisa memakai class StudenInfo dengan nama singkat nya.



```
}
@Override
public long adminFee(long amount) {
    return amount * StudentInfo.CARD_ADMIN_PERCENT/100;
}
```

Override adminFee di CreditCardPayment berguna agar kartu kredit memiliki biaya admin sendiri berupa persentase dari jumlah pembayaran, bukan memakai nilai default 0, sehingga setiap metode pembayaran dapat menghitung biayanya masing-masing tanpa mengubah kode CheckoutService.

- BankTransferPayment

```java
package shop.payment;
import shop.StudentInfo;
public class BankTransferPayment implements PaymentMethod {
    private final String bankName;

    public BankTransferPayment(String bankName) {
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    /** Simulasi: pembayaran berhasil selama jumlahnya positif. */
    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }
    @Override
    public long adminFee(long amount) {
        return StudentInfo.TRANSFER_ADMIN_FEE;
    }
}
```

```
abstraction-2511531014 > src > shop > payment >
package shop.payment;
import shop.StudentInfo;
```

Di lakukan penambahan dibagian import shop.StudentInfo dipakai supaya BankTransferPayment bisa memakai class StudenInfo dengan nama singkat nya.

```java
    @Override
    public long adminFee(long amount) {
        return StudentInfo.TRANSFER_ADMIN_FEE;
    }
}
```

Override adminFee di BankTransferPayment mengembalikan nilai tetap TRANSFER_ADMIN_FEE (Rp4.500) tanpa memperhatikan jumlah pembayaran, sehingga transfer bank memiliki biaya admin flat yang berbeda dengan kartu kredit yang berbasis persentase.

c. grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)

```java
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;

public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) { // a capability, not a class
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items, ShippingService shipping) {
        return subtotal(items) + shipping.calculateCost(totalWeight(items));
    }

    public long grandTotal(List<Product> items, ShippingService shipping, PaymentMethod payment) {
    long base = total(items, shipping);
    return base + payment.adminFee(base);
    }

    public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
    long totalAmount = grandTotal(items, shipping, payment);
    return payment.pay(totalAmount);
    }

    public boolean refund(RefundablePayment payment, long amount) {
    return payment.refund(amount);
    }
}
```

Dilakukan di Class CheckoutService, Menambahkan Method grandTotal dan mengubah Method Checkout pada bagian total menjadi grandTotal

```java
public long grandTotal(List<Product> items, ShippingService shipping, PaymentMethod payment) {
long base = total(items, shipping);
return base + payment.adminFee(base);
}

public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
long totalAmount = grandTotal(items, shipping, payment);
return payment.pay(totalAmount);
}
```

Method grandTotal ditambahkan di CheckoutService untuk menghitung total akhir yang harus dibayar, yaitu total belanja ditambah ongkir ditambah biaya admin dari metode pembayaran yang dipilih. Method checkout kemudian diubah agar memakai grandTotal sehingga pembayaran sudah memperhitungkan biaya admin. Karena parameternya bertipe interface PaymentMethod, CheckoutService tidak perlu mengetahui jenis pembayaran konkretnya.

d. Diamond policy(): kartu kredit menggabungkan kedua kebijakan

- RefundablePayment.java

Latihan > abstraction-2511531014 > src > shop > payment > RefundablePayment.java > ...

```java
package shop.payment;

public interface RefundablePayment extends PaymentMethod {
    boolean refund(long amount);

   default String policy() {
        return "Refund diperbolehkan";
    }
}

```

- InstallmentPayment.java

```java
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {

    int maxInstallmentMonths();

    /** Monthly amount, rounded up. */
    default long monthlyInstallment(long amount, int months) {
        if (months < 1 || months > maxInstallmentMonths()) {
            throw new IllegalArgumentException(
                    "Months must be between 1 and " + maxInstallmentMonths());
        }
        return (amount + months - 1) / months;
    }
    default String policy() {
    return "Cicilan tersedia";
    }
}
```

Setelah menambahkan code RefundablePayment dan InstallmentPayment, bagian CreditCardPayment memunculkan 1 error.

Penyebab munculnya eror tersebut adalah reditCardPayment meng-implement dua interface (RefundablePayment dan InstallmentPayment) yang masing-masing punya default method policy() dengan tanda tangan sama. Java tidak boleh memilih sendiri mana yang dipakai, karena bisa menghasilkan perilaku yang tidak diinginkan. Situasi ini disebut **diamond problem**, dan programmer harus menyelesaikannya secara

eksplisit. Untuk perbaikan tambahkan method policy pada class CreditCardPayment





Method policy() di-override di CreditCardPayment untuk menyelesaikan diamond problem, yaitu kondisi ketika sebuah class mewarisi dua default method bernama sama dari dua interface, dalam hal ini RefundablePayment dan InstallmentPayment. Compiler tidak boleh memilih salah satunya secara otomatis, sehingga programmer harus menentukan sendiri perilakunya. Dengan sintaks NamaInterface.super.policy(), kedua versi default dipanggil secara eksplisit lalu digabung menjadi satu teks, yaitu “Refund diperbolehkan; Cicilan tersedia”. Dengan cara ini kartu kredit menggabungkan kedua kebijakan, sedangkan EWalletPayment yang hanya meng-implement RefundablePayment tidak mengalami konflik dan otomatis memakai “Refund diperbolehkan”.

Jawab di laporan:

1. Mengapa EWalletPayment tidak perlu meng-override policy()?
   Jawab : EWalletPayment hanya meng-implement RefundablePayment, sehingga hanya mewarisi satu default policy() dan tidak ada konflik. Java otomatis memakai versi tersebut, jadi override tidak diperlukan.
2. Mengapa Java tidak memilih salah satu default secara otomatis?
   Jawab : Java tidak memilih otomatis karena CreditCardPayment mewarisi dua default method dengan tanda tangan sama dari dua interface yang sejajar, sehingga compiler tidak bisa menentukan mana yang dimaksud programmer. Memilih sendiri berisiko menimbulkan perilaku yang tidak diinginkan, jadi Java mewajibkan programmer menyelesaikannya secara eksplisit dengan meng-override method tersebut

e. TugasCheck

Setelah melakukan semua Langkah diatas lakukan uji coba di bagian tugasCheck melihat dibagian mana saja yang gagal atau lulus. Output yangg di hasil kan saat menjalankan Tugacheck:

```
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511531014 | Naysha Aprilia Rizki

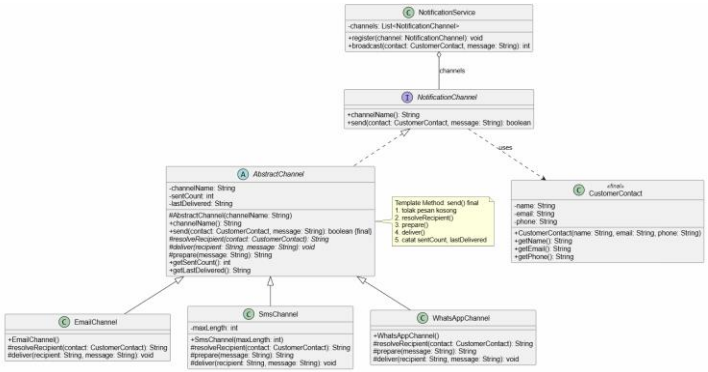
-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (2%, Rp4500, 114 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
```

Semua Tugas 1 berhasil di jalankan atau lulus.

3) Tugas 2 Merancang Sistem notifikasi

Rancang UML



a. NotificationChannel interface; AbstractChannel abstract dengan send(...) final

- NotificatonChannel

```java
package shop.notification;

public interface NotificationChannel {
    String channelName();
    boolean send(CustomerContact contact, String message);
}
```

Kode ini adalah interface NotificationChannel di package shop.notification yang berfungsi sebagai kontrak untuk semua kanal notifikasi. Interface ini memiliki dua method abstract, yaitu channelName() yang mengembalikan nama kanal, dan send(CustomerContact contact, String message) yang mengirim pesan ke kontak tertentu dan mengembalikan true jika pengiriman berhasil. Interface hanya menyatakan apa yang harus dapat dilakukan sebuah kanal tanpa menentukan caranya, sehingga setiap kanal, seperti email, SMS, WhatsApp, maupun anonymous class kanal log, bebas menentukan cara kirimnya sendiri. Nilai balik boolean pada send() dipakai oleh

NotificationService.broadcast() untuk menghitung jumlah kanal yang berhasil mengirim. Karena NotificationService hanya bergantung pada interface ini dan bukan pada kanal konkret, kanal baru dapat ditambahkan tanpa mengubah NotificationService (program to an interface).

- AbstractChannel



```java
package shop.notification;

public abstract class AbstractChannel implements NotificationChannel {
    private final String channelName;
    private int sentCount;
    private String lastDelivered = "";

    protected AbstractChannel(String channelName) {
        if (channelName == null || channelName.isBlank()) {
            throw new IllegalArgumentException("Channel name must not be blank");
        }
        this.channelName = channelName;
    }

    @Override
    public String channelName() {
        return channelName;
    }

    // urutan langkah dikunci
    @Override
    public final boolean send(CustomerContact contact, String message) {
        if (contact == null || message == null || message.isBlank()) {
            return false; // 1. Tolak pesan jika kosong
        }
        String recipient = resolveRecipient(contact); // 2. cari alamat penerima
        if (recipient == null) {
            return false; // 3.Tolak jika tidak ada alamat penerima
        }
        String prepared = prepare(message); // 4.Pesan disiapkan
        deliver(recipient, prepared); // 5.Pesan dikirim
        sentCount++; // 6.Hitung pesan yang dikirim
        lastDelivered = prepared;
        return true;
    }

    /**
     * @return alamat tujuan, atau null jika kontak tidak bisa dihubungi lewat kanal
     *         ini
     */
    protected abstract String resolveRecipient(CustomerContact contact);

    protected abstract void deliver(String recipient, String message);

    /** Hook: boleh di-override. Default: pesan apa adanya. */
    protected String prepare(String message) {
        return message;
    }

    public int getSentCount() {
        return sentCount;
    }

    public String getLastDelivered() {
        return lastDelivered;
    }

}
```

Kode ini adalah class abstract AbstractChannel yang meng-implement NotificationChannel dan menjadi kerangka bersama untuk semua kanal notifikasi. Class ini menyimpan nama kanal, jumlah pesan yang berhasil terkirim (sentCount), dan pesan terakhir yang terkirim (lastDelivered). Inti desainnya ada pada method send() yang bersifat final, yaitu mengunci urutan langkah pengiriman: menolak pesan atau kontak yang kosong, mencari alamat penerima lewat resolveRecipient(), menolak jika kanal ini tidak punya alamat untuk kontak tersebut, menyiapkan pesan lewat prepare(), mengirim lewat deliver(), lalu mencatat sentCount dan lastDelivered. Karena send() final, tidak ada kanal yang dapat mengubah urutan, melewati validasi, atau lupa mencatat pengiriman. Langkah yang berbeda antar kanal diserahkan kepada subclass: resolveRecipient() dan

deliver() bersifat protected abstract sehingga wajib diisi, sedangkan prepare() adalah method protected biasa (hook) yang secara default mengembalikan pesan apa adanya dan hanya di-override oleh kanal yang perlu mengubah pesan, seperti SmsChannel yang memotong dan WhatsAppChannel yang menambah awalan. Pola ini disebut Template Method.

b. EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel

- EmailChannel

```java
package shop.notification;

public class EmailChannel extends  AbstractChannel {

    public EmailChannel(){
        super(channelName: "Email");
    }


    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String email = contact.getEmail();
        return email.contains("@") ? email : null;
    }
    @Override
    protected void deliver(String recipient, String message) {
        System.out.println("[EMAIL to " + recipient + "] " + message);
    }
}
```

Kode ini adalah class EmailChannel yang merupakan subclass dari AbstractChannel dan mewakili kanal notifikasi lewat email. Constructor-nya tanpa parameter dan memanggil super("Email") untuk memberi nama kanal. Class ini hanya mengisi dua langkah yang berbeda antar kanal sesuai kerangka Template Method di AbstractChannel. Pertama, resolveRecipient() mengambil email dari kontak dan mengembalikannya hanya jika memuat karakter @, sedangkan jika tidak, ia mengembalikan null sehingga send() di AbstractChannel menolak pengiriman. Kedua, deliver() mensimulasikan pengiriman dengan mencetak pesan beserta alamat tujuan ke konsol. EmailChannel tidak meng-override send() yang final dan tidak meng-override prepare(), sehingga pesan dikirim apa adanya dan urutan langkah, pencatatan sentCount, serta lastDelivered tetap ditangani oleh AbstractChannel.

- SmsChannel



```java
package shop.notification;

public class SmsChannel extends AbstractChannel {

    private static final String ELLIPSIS = "...";
    private final int maxLength;

    public SmsChannel(int maxLength) {
        super(channelName: "SMS");
        if (maxLength <= ELLIPSIS.length()) {
            throw new IllegalArgumentException("maxLength must be greater than 3");

        }
        this.maxLength = maxLength;
    }

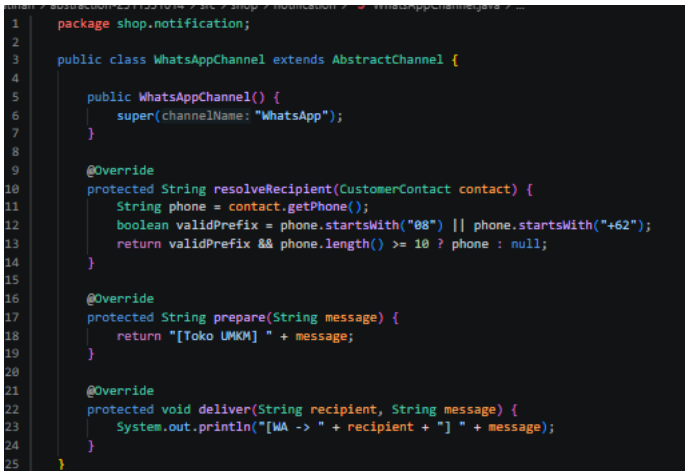
    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String phone = contact.getPhone();
        return phone.startsWith("08") ? phone: null;
    }

    @Override
    protected String prepare(String message) {
        if (message.length() <= maxLength) {
            return message;
        }
        return message.substring(0, maxLength - ELLIPSIS.length()) + ELLIPSIS;
    }

    @Override
    protected void deliver(String recipient, String message) {
        System.out.println("[SMS -> " + recipient + "] " + message);
    }
}
```

Kode ini adalah class SmsChannel yang merupakan subclass dari AbstractChannel dan mewakili kanal notifikasi lewat SMS. Constructor-nya menerima maxLength, yaitu batas panjang pesan (diisi dengan StudentInfo.SMS_MAX_LENGTH = 114), memanggil super(“SMS”) untuk memberi nama kanal, dan melempar IllegalArgumentException jika maxLength tidak lebih dari 3 karena harus cukup untuk memuat tanda “...”. Class ini mengisi tiga langkah dari kerangka Template Method. resolveRecipient() mengambil nomor telepon kontak dan mengembalikannya hanya jika diawali “08”, sehingga nomor berformat +62 ditolak. prepare() di-override untuk memotong pesan yang lebih panjang dari maxLength menjadi tepat maxLength karakter, yaitu maxLength dikurangi 3 karakter pertama pesan ditambah “...”, sedangkan pesan yang tidak melebihi batas dikirim apa adanya. deliver() mensimulasikan pengiriman dengan mencetak pesan beserta nomor tujuan ke konsol. SmsChannel tidak meng-override send(), sehingga urutan langkah tetap dikunci oleh AbstractChannel.

- WhatAppChannel



```java
package shop.notification;

public class WhatsAppChannel extends AbstractChannel {

    public WhatsAppChannel() {
        super(channelName: "WhatsApp");
    }

    @Override
    protected String resolveRecipient(CustomerContact contact) {
        String phone = contact.getPhone();
        boolean validPrefix = phone.startsWith("08") || phone.startsWith("+62");
        return validPrefix && phone.length() >= 10 ? phone : null;
    }

    @Override
    protected String prepare(String message) {
        return "[Toko UMKM] " + message;
    }

    @Override
    protected void deliver(String recipient, String message) {
        System.out.println("[WA -> " + recipient + "] " + message);
    }
}
```

Kode ini adalah class WhatsAppChannel yang merupakan subclass dari AbstractChannel dan mewakili kanal notifikasi lewat WhatsApp. Constructor-nya tanpa parameter dan memanggil super("WhatsApp") untuk memberi nama kanal. Class ini mengisi tiga langkah dari kerangka Template Method. resolveRecipient() mengambil nomor telepon kontak dan mengembalikannya hanya jika nomor diawali "08" atau "+62" dan panjangnya minimal 10 karakter, sedangkan jika tidak memenuhi, ia mengembalikan null sehingga send() menolak pengiriman. Berbeda dengan SmsChannel, kanal ini juga menerima nomor berformat internasional +62. prepare() di-override untuk menambahkan awalan "[Toko UMKM] " di depan setiap pesan, dan hasilnya inilah yang dicatat sebagai lastDelivered. deliver() mensimulasikan pengiriman dengan mencetak pesan beserta nomor tujuan ke konsol. WhatsAppChannel tidak meng-override send(), sehingga urutan langkah tetap dikunci oleh AbstractChannel.

c. SmsChannel memotong pesan panjang tepat 114 karakter berakhiran "..."

```
    private static final String ELLIPSIS = "...";
    private final int maxLength;

    public SmsChannel(int maxLength) {
        super(channelName: "SMS");
        if (maxLength <= ELLIPSIS.length()) {
            throw new IllegalArgumentException("maxLength must be greater than 3");

        }
        this.maxLength = maxLength;
    }
```

```
@Override
protected String prepare(String message) {
    if (message.length() <= maxLength) {
        return message;
    }
    return message.substring(0, maxLength - ELLIPSIS.length()) + ELLIPSIS;
}
```

Pemotongan pesan SMS dilakukan oleh method prepare() di SmsChannel, yang dipanggil otomatis oleh send() di AbstractChannel pada langkah menyiapkan pesan. Batas panjang disimpan di field maxLength lewat constructor, dan diisi 114 dari StudentInfo.SMS_MAX_LENGTH sesuai NIM. Konstanta ELLIPSIS menyimpan teks "..." yang panjangnya 3 karakter. Jika panjang pesan tidak melebihi maxLength, pesan dikirim apa adanya. Jika lebih panjang, method mengambil 111 karakter pertama (maxLength dikurangi panjang ELLIPSIS, yaitu 114 − 3) lalu menambahkan "...", sehingga hasil akhirnya tepat 114 karakter dan berakhiran "...". Constructor juga menolak maxLength yang tidak lebih dari 3 agar pengurangan tersebut tidak menghasilkan nilai nol atau negatif. Hasil dari prepare() inilah yang dikirim oleh deliver() dan dicatat sebagai lastDelivered

d. EmailChannel menolak kontak tanpa email yang valid

Bagian yang bertugas untuk menolak kontak tanpa emal yang valiad ada pada EmailChannel dan AbstractChannel

- Email Channel

```
@Override
protected String resolveRecipient(CustomerContact contact) {
    String email = contact.getEmail();
    return email.contains("@") ? email : null;
}
```

- Abstract Channel

```
String recipient = resolveRecipient(contact); // 2. cari alamat penerima
if (recipient == null) {
    return false; // 3.Tolak jika tidak ada alamat penerima
}
```

Penolakan kontak tanpa email yang valid ditangani oleh dua bagian kode yang bekerja sama. Pertama, method resolveRecipient() di EmailChannel mengambil email dari kontak dan mengembalikannya hanya jika memuat karakter "@"; jika email kosong atau tidak memuat "@", method mengembalikan null sebagai tanda bahwa kontak tersebut tidak dapat dihubungi lewat kanal email. Kedua, method send() di AbstractChannel yang bersifat final memeriksa hasil tersebut: jika recipient bernilai null, send() langsung mengembalikan false sebelum prepare(), deliver(), dan pencatatan dijalankan, sehingga pesan tidak dikirim dan sentCount tidak bertambah. Dengan demikian kontak tanpa email, misalnya dengan email kosong, ditolak, sedangkan kontak dengan email "uji@example.com" diterima dan sentCount menjadi 1.

e. NotificationService.broadcast(...) mengembalikan jumlah kanal yang berhasil

```
1  package shop.notification;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class NotificationService {
7
8      private final List<NotificationChannel> channels = new ArrayList<>();
9
10     public void register(NotificationChannel channel) {
11         if (channel == null) {
12             throw new IllegalArgumentException("Channel must not be null");
13         }
14         channels.add(channel);
15     }
16
17     public int broadcast(CustomerContact contact, String message) {
18         int delivered = 0;
19         for (NotificationChannel channel : channels) {
20             if (channel.send(contact, message)) {
21                 delivered++;
22             }
23         }
24         return delivered;
25     }
26 }
```

NotificationService mengelola sekumpulan kanal notifikasi dan bergantung pada interface NotificationChannel, bukan pada kanal konkret. Method register() menambahkan kanal ke daftar channels dan menolak nilai null dengan IllegalArgumentException. Method

broadcast() mengirim satu pesan ke satu kontak lewat semua kanal yang terdaftar: ia mengulang setiap kanal, memanggil send(), dan menambah penghitung delivered setiap kali send() mengembalikan true, lalu mengembalikan jumlah kanal yang berhasil. Kanal yang menolak kontak, misalnya email untuk kontak tanpa alamat email atau SMS untuk nomor berawalan +62, mengembalikan false sehingga tidak dihitung dan tidak menghentikan kanal lain. Pada contoh TugasApp, Siti yang memiliki email dan nomor 08 diterima oleh 4 kanal, sedangkan Budi yang hanya memiliki nomor +62 diterima oleh 2 kanal (WhatsApp dan log). Karena hanya bergantung pada interface, kanal baru, termasuk anonymous class kanal log, dapat didaftarkan tanpa mengubah NotificationService.

f. Demo (Buat src/shop/app/TugasApp.java)

1. menampilkan grand total keranjang contoh (kurir Ekspres) untuk tiga metode pembayaran;
2. mencetak policy() kartu kredit dan e-wallet;
3. mendaftarkan ketiga kanal (SMS memakai StudentInfo.SMS_MAX_LENGTH) dan satu kanal log konsol berupa anonymous class NotificationChannel;
4. mengirim satu pesan panjang (lebih dari 200 karakter) ke dua kontak: satu punya email dan nomor 08..., satu hanya punya nomor +62....

TugasApp.Java

- Package dan Import

```java
package shop.app;

import java.util.List;

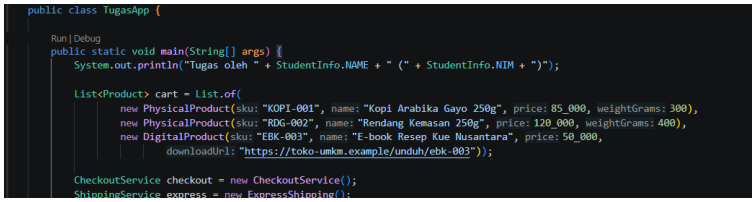
import shop.StudentInfo;
import shop.model.DigitalProduct;
import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.notification.CustomerContact;
import shop.notification.EmailChannel;
import shop.notification.NotificationChannel;
import shop.notification.NotificationService;
import shop.notification.SmsChannel;
import shop.notification.WhatsAppChannel;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.EWalletPayment;
import shop.payment.PaymentMethod;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.ShippingService;

```

Bagian ini menyatakan bahwa TugasApp berada di package shop.app, lalu mengimpor class yang dipakai dari package lain karena Java hanya mengenali class satu package secara otomatis. Import dikelompokkan menurut package asalnya: shop.StudentInfo untuk data NIM, shop.model untuk produk keranjang, shop.notification untuk kontak, kanal, dan NotificationService, shop.payment untuk metode pembayaran, shop.service untuk CheckoutService, dan shop.shipping untuk kurir. Tipe

interface seperti PaymentMethod, NotificationChannel, dan ShippingService ikut diimpor karena dipakai sebagai tipe variabel, sesuai prinsip memprogram ke interface.

- Persiapan data



```
public class TugasApp {

    Run | Debug
    public static void main(String[] args) {
        System.out.println("Tugas oleh " + StudentInfo.NAME + " (" + StudentInfo.NIM + ")");

        List<Product> cart = List.of(
                new PhysicalProduct(sku: "KOPI-001", name: "Kopi Arabika Gayo 250g", price: 85_000, weightGrams: 300),
                new PhysicalProduct(sku: "RDG-002", name: "Rendang Kemasan 250g", price: 120_000, weightGrams: 400),
                new DigitalProduct(sku: "EBK-003", name: "E-book Resep Kue Nusantara", price: 50_000,
                        downloadUrl: "https://toko-umkm.example/unduh/ebk-003"));

        CheckoutService checkout = new CheckoutService();
        ShippingService express = new ExpressShipping();
```

Bagian ini menyiapkan bahan untuk seluruh demo. Program mencetak nama dan NIM dari class StudentInfo, lalu membuat keranjang contoh berisi dua produk fisik (kopi dan rendang) dan satu produk digital (e-book). Dibuat pula objek CheckoutService untuk menghitung total, dan ExpressShipping sebagai kurir. Keranjang bertipe List<Product> sehingga bisa memuat produk fisik maupun digital sekaligus.

- Grand Total dan 3 metode pembayaran



```
        // 1. Grand total keranjang (ekspres) untuk tiga metode pembayaran
        System.out.println("\n== T1: Total dengan biaya admin (ekspres) ==");
        EWalletPayment wallet = new EWalletPayment(name: "E-Wallet DompetKu", balance: 1_000_000);
        PaymentMethod transfer = new BankTransferPayment(bankName: "Bank Nagari");
        CreditCardPayment card = new CreditCardPayment(cardHolder: "Siti Rahma", creditLimit: 1_000_000);

        PaymentMethod[] methods = {wallet, transfer, card};
        for (PaymentMethod method : methods) {
            long base = checkout.total(cart, express);
            System.out.println(method.getName() + " : Rp"
                    + checkout.grandTotal(cart, express, method)
                    + " (admin Rp" + method.adminFee(base) + ")");
        }
```

Tiga metode pembayaran dibuat, yaitu EWalletPayment, BankTransferPayment, dan CreditCardPayment, lalu disimpan dalam array bertipe PaymentMethod dan diproses dengan satu perulangan. Untuk tiap metode, program memanggil checkout.grandTotal() untuk total akhir dan method.adminFee() untuk biaya admin. Karena tipe yang dipakai adalah interface, kode yang sama menghasilkan perilaku berbeda: e-wallet memakai adminFee() default sebesar Rp0, transfer bank memakai biaya tetap Rp4.500, dan kartu kredit memakai 2% dari total sebesar Rp5.360. Inilah polimorfisme, yaitu CheckoutService tidak perlu mengetahui jenis pembayarannya

- Policy () kartu kredit dan E-wallet



```
        // 2. policy() kartu kredit dan e-wallet
        System.out.println();
        System.out.println("Kebijakan kartu kredit : " + card.policy());
        System.out.println("Kebijakan e-wallet     : " + wallet.policy());
```

Program mencetak policy() dari kartu kredit dan e-wallet. E-wallet hanya meng-implement RefundablePayment sehingga otomatis memakai default “Refund maksimal 7 hari”. Kartu kredit meng-implement

RefundablePayment dan InstallmentPayment yang sama-sama memiliki default policy(), sehingga CreditCardPayment meng-override-nya dan menggabungkan keduanya dengan sintaks NamaInterface.super.policy(). Hasilnya “Refund maksimal 7 hari; Cicilan maksimal 12 bulan”. Ini adalah penyelesaian diamond problem pada T1.e.

- Mendaftarkan 4 kanal



```
// 3. Daftarkan tiga kanal + satu kanal log (anonymous class)
System.out.println("\n== T2: Notifikasi status pesanan ==");
NotificationService service = new NotificationService();
service.register(new EmailChannel());
service.register(new SmsChannel(StudentInfo.SMS_MAX_LENGTH));
service.register(new WhatsAppChannel());
service.register(new NotificationChannel() {
    @Override
    public String channelName() {
        return "Log Konsol";
    }

    @Override
    public boolean send(CustomerContact contact, String message) {
        System.out.println("[LOG] " + contact.getName() + ": " + message);
        return true;
    }
});
```

Program membuat NotificationService lalu mendaftarkan EmailChannel, SmsChannel, dan WhatsAppChannel. SmsChannel menerima StudentInfo.SMS_MAX_LENGTH (114) sebagai batas panjang pesan sesuai NIM. Kanal keempat berupa anonymous class yang langsung meng-implement NotificationChannel dan hanya mencetak pesan dengan awalan [LOG], tanpa membuat class bernama baru dan tanpa mengubah NotificationService. Hal ini dimungkinkan karena NotificationService menyimpan daftar bertipe interface NotificationChannel, bukan kanal konkret tertentu.

- Mengirim pesan ke dua kontak



```
// 4. Satu pesan panjang (> 200 karakter) ke dua kontak
String message = "Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. "
        + "Nomor resi akan muncul di aplikasi dalam 1x24 jam. "
        + "Terima kasih telah berbelanja produk UMKM Sumatera Barat di toko kami! "
        + "Hubungi kami jika pesanan belum sampai dalam 3 hari kerja.";

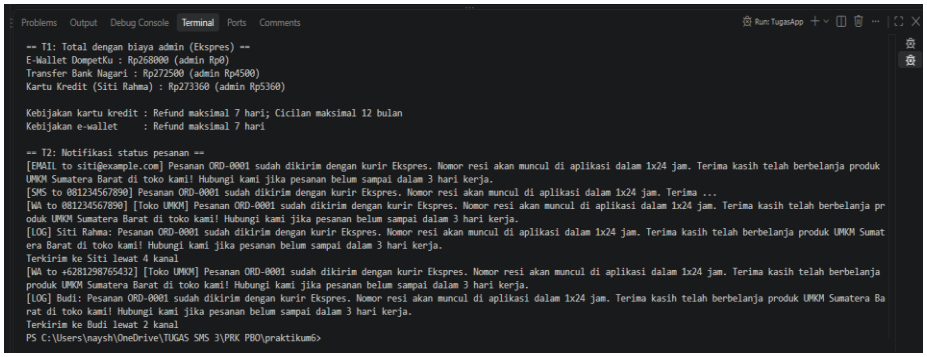
CustomerContact siti = new CustomerContact(name: "Siti Rahma", email: "siti@example.com", phone: "081234567890");
CustomerContact budi = new CustomerContact(name: "Budi", email: "", phone: "+6281298765432");

for (CustomerContact contact : List.of(siti, budi)) {
    int delivered = service.broadcast(contact, message);
    System.out.println("Terkirim ke " + contact.getName().split(" ")[0]
            + " lewat " + delivered + " kanal");
}
}
```

atu pesan yang lebih dari 200 karakter dikirim lewat service.broadcast() ke dua kontak. Siti (email dan nomor 08...) diterima oleh keempat kanal: email mengirim apa adanya, SMS memotong pesan menjadi tepat 114 karakter berakhiran “...”, WhatsApp menambah awalan “[Toko UMKM] “, dan kanal log mencetak pesan. Budi (hanya nomor +62...) hanya diterima oleh dua kanal: email menolak karena tidak ada alamat email, SMS menolak karena nomornya tidak berawalan 08, sedangkan WhatsApp menerima nomor +62 dan kanal log menerima semua kontak. Nilai kembalian broadcast() adalah jumlah kanal yang berhasil, yaitu 4

untuk Siti dan 2 untuk Budi, dan nilai ini dicetak sebagai “Terkirim ke ... lewat ... kanal”.

- Output



Output di atas membuktikan TugasApp berjalan sesuai spesifikasi. Bagian T1 menampilkan total belanja dengan kurir Ekspres untuk tiga metode pembayaran: e-wallet Rp268.000 (admin Rp0), transfer bank Rp272.500 (admin tetap Rp4.500), dan kartu kredit Rp273.360 (admin 2% sebesar Rp5.360). Perbedaan biaya admin muncul karena tiap metode meng-override adminFee() dengan caranya sendiri, sedangkan e-wallet memakai default Rp0. Dua baris kebijakan menunjukkan hasil penyelesaian diamond problem: kartu kredit menggabungkan “Refund maksimal 7 hari; Cicilan maksimal 12 bulan”, sedangkan e-wallet hanya “Refund maksimal 7 hari”.

Bagian T2 menampilkan satu pesan yang dikirim ke dua kontak. Untuk Siti, pesan diterima email, SMS, WhatsApp, dan kanal log, sehingga tercetak “Terkirim ke Siti lewat 4 kanal”. Pesan SMS terpotong tepat 114 karakter berakhiran “...”, sedangkan pesan WhatsApp diberi awalan “[Toko UMKM] “. Untuk Budi yang hanya memiliki nomor +62, hanya WhatsApp dan kanal log yang berhasil, sehingga tercetak “Terkirim ke Budi lewat 2 kanal”, karena email menolak (tidak ada alamat email) dan SMS menolak (nomor tidak berawalan 08). Angka biaya admin transfer dan batas SMS berbeda dari contoh di modul karena mengikuti NIM 2511531014.

Jawab di laporan:

1. Mengapa ada interface dan abstract class, bukan salah satunya saja?

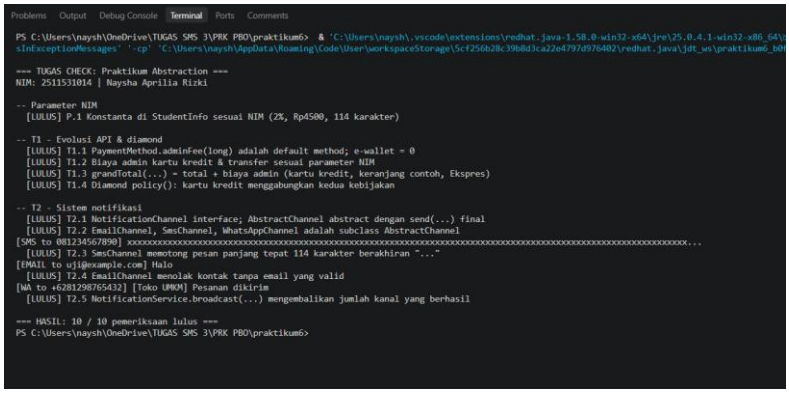
Jawab: Interface menjadi kontrak sehingga kanal apa pun (termasuk anonymous class) bisa didaftarkan, sedangkan abstract class menyediakan kode bersama (state dan kerangka send()) agar tidak ditulis

ulang di tiap kanal. Hanya interface membuat kode berulang, hanya abstract class memaksa semua kanal mewarisi satu superclass.

2. Apa akibatnya jika send(...) tidak dibuat final?
Jawab: Subclass bisa mengubah urutan langkah, melewati validasi, atau lupa mencatat sentCount, sehingga perilaku antar kanal tidak konsisten. Dengan final, urutan terkunci dan subclass hanya mengisi langkah yang disediakan.

3. Mengapa NotificationService menyimpan List, bukan List?
Jawab: Karena NotificationService hanya butuh send(), yang dijamin oleh interface. Dengan List<AbstractChannel>, anonymous class kanal log tidak bisa didaftarkan karena tidak mewarisi AbstractChannel.

g. TugasCheck
Setelah melakukan semua Langkah diatas lakukan uji coba di bagian tugasCheck melihat dibagian mana saja yang gagal atau lulus. Output yangg di hasil kan saat menjalankan Tugacheck:


Problems Output Debug Console Terminal Ports Comments
PS C:\Users\naysh\OneDrive\TUGAS SMS 3\PRK PBO\praktikum6> & 'C:\Users\naysh\.vscode\extensions\redhat.java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\
sInExceptionMessages' '-cp' 'C:\Users\naysh\AppData\Roaming\Code\User\workspaceStorage\5cf256b28c39b8d3ca22e4797d976402\redhat.java\jdt_ws\praktikum6_b0
=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511531014 | Naysha Aprilia Rizki
-- Parameter NIM
[LULUS] P.1 Konstanta di StudentInfo sesuai NIM (2%, Rp4500, 114 karakter)
-- T1 - Evolusi API & diamond
[LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
[LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
[LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
[LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
-- T2 - Sistem notifikasi
[LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
[LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS to 081234567890] xxxx

Permasalahan, ada ongkir yang bernilai 0, alamat pesanan yang berubah sendiri, dan program yang berhenti dengan exception.

b. Masalah Desain

No	Lokasi(class/method)	Masalah	Dampak	Perbaikan
1.	interface StoreOperations, GiftVoucher	**Interface terlalu gemuk**: voucher dipaksa meng-implement refund() dan ship() yang tidak relevan	ship() melempar UnsupportedOperationException saat dijalankan, program crash. Compiler tidak memperingatkan	Pecah jadi interface kecil (Payable, Refundable, Printable), voucher hanya meng-implement yang ia bisa
2.	Courier.cost(String type, int kg)	**switch pada string** menggantikan polimorfisme/enum	Typo "EXPRES" jatuh ke default dan menghasilkan ongkir **0** tanpa error. Jenis pengiriman baru harus mengubah method lama	Pakai enum ShippingType dengan tarif di dalamnya
3.	Address dan SimpleOrder	Class mutable (ada setter, tidak final) dibagi dua objek	Mengubah kota ORD-2 ikut mengubah ORD-1 karena memakai objek Address yang sama	Address dibuat final, field final, tanpa setter, perubahan lewat withCity()
4.	Cart.Item	Inner class yang tidak butuh objek luar	Tiap Item menyimpan referensi tersembunyi ke Cart (boros dan menahan objek tetap hidup), dan pembuatannya harus cart.new Item(...)	Jadikan static nested class
5.	Discount.apply()	**instanceof di dalam default method** interface	Interface bergantung pada implementornya. Diskon baru harus	Tiap diskon menyatakan persentasenya sendiri

			mengubah interface, dan lupa menambah cabang membuat diskon diam-diam tidak berlaku (harga penuh)	(percent()), apply() memakai itu
6.	Courier dan JneCourier	**Abstract class tanpa abstract method, subclass kosong**	Struktur pewarisan tidak memberi manfaat dan menyesatkan pembaca	Jadikan satu class Courier, atau interface bila memang ada banyak kurir
7.	GiftVoucher.balance	**Uang bertipe double**	Output 70000.0 dan berisiko error pembulatan pecahan	Pakai long seperti bagian lain program

c. file baru src/review/fixed/BadShopRefactored.java
Pada bagian ini saya memilih 3 perbaikan yaitu
1. Interface gemuk
2. Switch string jadi enum
3. Address mutable jadi immutable

Penjelasan

- Package dan import

```
package review.fixed;

import java.util.ArrayList;
import java.util.List;
```

package review.fixed; menandakan file ini ada di folder src/review/fixed/, jadi class-nya terpisah dari BadShop.java dan tidak bentrok nama. ArrayList dan List dipakai oleh Cart untuk menyimpan daftar item.

- Interface gemuk di pecah
Sebelum:

```java
interface StoreOperations {
    boolean pay(long amount);

    boolean refund(long amount);

    long ship(int weightGrams);

    String printInvoice();
}

class GiftVoucher implements StoreOperations {
    private double balance = 100000.0;

    @Override
    public boolean pay(long amount) {
        if (amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }
```

Setelah:

```java
// ---------- Masalah 1: interface dipecah ----------
interface Payable {
    boolean pay(long amount);
}

interface Refundable {
    boolean refund(long amount);
}

interface Printable {
    String printInvoice();
}

class GiftVoucher implements Payable, Printable {
    private double balance = 100_000;

    @Override
    public boolean pay(long amount) {
        if (amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }

    @Override
    public String printInvoice() {
        return "Voucher, remaining balance " + balance;
    }
}
```

StoreOperations memaksa GiftVoucher mengimplementasikan refund() dan ship() padahal voucher tidak bisa di-refund maupun dikirim, sehingga dua method itu hanya melempar UnsupportedOperationException dan program crash saat dipanggil. Interface dipecah menjadi Payable, Refundable, dan Printable (Interface Segregation Principle), sehingga voucher hanya mengimplementasikan kemampuan yang benar-benar dimilikinya. Pemanggilan ship() pada voucher kini ditolak compiler, bukan crash saat program berjalan.

Cara kerja: Satu interface besar dipecah menjadi tiga interface kecil, masing-masing satu kemampuan. GiftVoucher hanya menandatangani Payable dan Printable, jadi tidak ada lagi method palsu refund() dan ship(). Kalau ada yang menulis

voucher.ship(100), compiler langsung menolak. Refundable disiapkan untuk metode bayar lain yang memang bisa di-refund.

- Switch string jadi enum

  Sebelum:

```
abstract class Courier {
    public long cost(String type, int kilograms) {
        switch (type) {
            case "REGULAR":
                return kilograms * 10_000L;
            case "EXPRESS":
                return kilograms * 18_000L;
            case "SAME_DAY":
                return kilograms * 30_000L;
            default:
                return 0;
        }
    }
}
```

  Setelah:

```
// ---------- Masalah 2: enum menggantikan switch pada String ----------
enum ShippingType {
    REGULAR(costPerKg: 10_000),
    EXPRESS(costPerKg: 18_000),
    SAME_DAY(costPerKg: 30_000);

    private final long costPerKg;

    ShippingType(long costPerKg) {
        this.costPerKg = costPerKg;
    }

    long costPerKg() {
        return costPerKg;
    }
}

abstract class Courier {
    public long cost(ShippingType type, int kilograms) {
        return kilograms * type.costPerKg();
    }
}

class JneCourier extends Courier {
}
```

  Jenis pengiriman semula berupa String yang dicek dengan switch, sehingga typo seperti “EXPRES” tidak menimbulkan error dan ongkir diam-diam bernilai 0. Dengan enum ShippingType, nilai yang valid dibatasi compiler, jadi typo langsung gagal saat kompilasi.

Tarif tiap jenis juga disimpan di satu tempat, sehingga menambah jenis baru tidak perlu mengubah switch.

Cara kerja: Tarif per kilogram dipindah ke dalam enum, dan REGULAR(10_000) memanggil constructor enum untuk menyimpannya di costPerKg. Parameter cost() berubah dari String menjadi ShippingType, sehingga switch dan default: return 0 tidak diperlukan lagi. Contoh: EXPRESS 2 kg dihitung 2 * 18000 = 36000.

- Address mutable jadi immutable
  Sebelum:

```
class Address {
    private String street;
    private String city;

    Address(String street, String city) {
        this.street = street;
        this.city = city;
    }

    public void setStreet(String street) {
        this.street = street;
    }

    public void setCity(String city) {
        this.city = city;
    }

    @Override
    public String toString() {
        return street + ", " + city;
    }
}
```

  Setelah:



```
// ---------- Masalah 3: Address immutable ----------
final class Address {
    private final String street;
    private final String city;

    Address(String street, String city) {
        this.street = street;
        this.city = city;
    }

    Address withCity(String newCity) {
        return new Address(street, newCity);
    }

    @Override
    public String toString() {
        return street + ", " + city;
    }
}

class SimpleOrder {
    private final String id;
    private final Address address;

    SimpleOrder(String id, Address address) {
        this.id = id;
        this.address = address;
    }

    @Override
    public String toString() {
        return id + " -> " + address;
    }
}
```

Address semula mutable dan objek yang sama dipakai oleh dua order, sehingga mengubah kota ORD-2 ikut mengubah ORD-1. Class dibuat final, semua field final, setter dihapus, dan perubahan dilakukan lewat withCity() yang membuat objek baru, sehingga objek yang dibagi tidak bisa berubah diam-diam.

Cara kerja: final class mencegah pewarisan, private final membuat field tidak bisa diubah setelah objek dibuat, dan setter dihapus. withCity() tidak mengubah objek lama, melainkan membuat Address baru. Di main, home tetap Padang untuk ORD-1, sedangkan ORD-2 memakai objek baru berkota Bukittinggi.

- Untuk kesalahan 4,5,6,7 ga dilakukan perubahan tetap seperti awal

```java
// ---------- Tidak diubah (masalah 4, 5, 6, 7 dibiarkan) ----------
class Cart {
    private final List<Item> items = new ArrayList<>();

    class Item {
        private final String name;
        private final long price;
        private final int quantity;

        Item(String name, long price, int quantity) {
            this.name = name;
            this.price = price;
            this.quantity = quantity;
        }

        long subtotal() {
            return price * quantity;
        }

        String getName() {
            return name;
        }
    }

    void add(Item item) {
        items.add(item);
    }

    long total() {
        long total = 0;
        for (Item item : items) {
            total += item.subtotal();
        }
        return total;
    }
}
```

```java
interface Discount {
    default long apply(long price) {
        if (this instanceof MemberDiscount) {
            return price - price * 5 / 100;
        } else if (this instanceof FlashSaleDiscount) {
            return price - price * 30 / 100;
        }
        return price;
    }
}

class MemberDiscount implements Discount {
}

class FlashSaleDiscount implements Discount {
}

public class BadShopRefactored {
    Run | Debug
    public static void main(String[] args) {
        Cart cart = new Cart();
        cart.add(cart.new Item(name: "Kopi Gayo", price: 85_000, quantity: 2));
        System.out.println("Total cart: " + cart.total());

        Courier courier = new JneCourier();
        System.out.println("Ongkir EXPRESS 2 kg: " + courier.cost(ShippingType.EXPRESS, kilograms: 2));

        Address home = new Address(street: "Jl. Sudirman 10", city: "Padang");
        SimpleOrder first = new SimpleOrder(id: "ORD-1", home);
        SimpleOrder second = new SimpleOrder(id: "ORD-2", home.withCity(newCity: "Bukittinggi"));
        System.out.println(first);
        System.out.println(second);
    }
}
```

Ouput :

```
PS C:\Users\naysh\OneDrive\TUGAS SMS 3\PRK PBO\praktikum6>  c:; cd 'c:\Users\naysh\OneDrive\
java-1.58.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExcepti
5cf256b28c39b8d3ca22e4797d976402\redhat.java\jdt_ws\praktikum6_b0f9e960\bin' 'review.fixed.B
Total cart: 170000
Ongkir EXPRESS 2 kg: 36000
ORD-1 -> Jl. Sudirman 10, Padang
ORD-2 -> Jl. Sudirman 10, Bukittinggi
PS C:\Users\naysh\OneDrive\TUGAS SMS 3\PRK PBO\praktikum6>
```

5) Tugas 4 Refleksi

1. Abstrac class vs Interface

Pertanyaan Desain	Abstract class/Interface	Alasan	Contoh dari Proyek
Subclass berbagi field dan konstruktor	Abstract class	Interface tidak bisa menyimpan field milik objek dan tidak punya	AbstractChannel menyimpan channelName, sentCount, dan lastDelivered, lalu diisi

		konstruktor. Abstract class bisa, jadi kode yang sama tidak perlu ditulis ulang di tiap subclas	lewat konstruktor super("Email"), super("SMS"), dan super("WhatsApp").
Satu class perlu beberapa kemampuan sekaligus	Interface	Java hanya membolehkan extends satu class, tapi implements boleh banyak interface. Kemampuan lebih cocok dibuat interface supaya bisa digabung.	CreditCardPayment implements RefundablePayment, InstallmentPayment, jadi kartu kredit bisa refund dan cicilan sekaligus.
Rumus bersama yang tidak boleh diubah subclass	**Abstract class** (dengan method final)	Method di abstract class bisa dikunci final. Interface tidak bisa mengunci method default karena subclass tetap bisa meng-override-nya.	AbstractChannel.send() dibuat final dengan urutan tolak pesan kosong, cari tujuan, siapkan, kirim, lalu catat. Email, SMS, dan WA hanya mengisi resolveRecipient, prepare, dan deliver.
Kontrak yang perlu ditambah method baru di kemudian hari	**Interface** dengan default method	Menambah method abstract memaksa semua class pengimplementasi diubah. Method default punya isi bawaan, jadi class lama tetap jalan.	adminFee() ditambahkan ke PaymentMethod. Saat masih abstract muncul 4 error, termasuk di voucher anonymous class ShopApp. Setelah dijadikan default (return 0), EWalletPayment dan voucher tidak perlu diubah.

2. Jenis nested class

Kondisi	Static nested,inner atau annoymous	Alasan	Contoh dari Proyek
Class pembantu tidak membaca data objek luar	Static nested	Static nested tidak menyimpan referensi	Cart.Item di BadShop (temuan 4) hanya memakai field

		tersembunyi ke objek luar, jadi lebih ringan dan bisa dibuat tanpa objek luar: new Cart.Item(...).	miliknya sendiri (name, price, quantity), tidak ada field Cart yang dibaca. Seharusnya static
Objek harus membaca field objek luarnya	Innerclass	Inner class otomatis memegang referensi ke objek luar, sehingga bisa langsung membaca field-nya, termasuk yang private.	
Implementasi sekali pakai, tidak perlu nama	Annoymous class	Dipakai di satu tempat saja, jadi tidak perlu membuat file atau class bernama.	Kanal "Log Konsol" di TugasApp (new NotificationChannel() { ... }) dan voucher di ShopApp.